

A
PROJECT REPORT
ON
MINDLAB – SCIENCE MEMORY CHALLENGE

A web-based memory training game that teaches science facts

Submitted in partial fulfilment of the requirements
for the award of the degree of
BACHELOR OF COMPUTER APPLICATIONS (BCA)

Submitted To:

[Name of Project Guide]

HOD, BCA Deptt.

Submitted By:

[Student Name]

BCA 5th Sem

Roll No: [.....]

[NAME OF COLLEGE / INSTITUTE]

[Affiliated to: Name of University]

Session: [20__ – 20__]

TABLE OF CONTENTS

Content	Page No.
1. Declaration	3
2. Certificate	4
3. Acknowledgement	5
4. Abstract	6
5. Introduction of the Project	7
6. Objectives of the Project	10
7. Hardware and Software Requirements	11
8. Project Analysis & Designing	13
- DFD	17
- ER Diagram	21
9. Modules	26
- Process logic of each module	26
10. Input / Output Screenshot	33
11. Coding	42
12. Testing	60
13. Future Scope	66
14. References	68

DECLARATION

I, [Student Name], a student of BCA 5th Semester, Roll No. [...], [Name of College], hereby declare that the project report entitled “**MindLab – Science Memory Challenge**” is submitted by me in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications.

This report is a record of the project work carried out by me under the guidance of [Name of Project Guide]. The work presented in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Place: [...]

Date: [...]

(Signature of the Student)

[Student Name]

BCA 5th Sem

Roll No: [...]

CERTIFICATE

This is to certify that the project report entitled “**MindLab – Science Memory Challenge**” is a bonafide record of the project work carried out by **[Student Name]**, Roll No. **[.....]**, a student of BCA 5th Semester, **[Name of College]**.

The project has been completed under my supervision and guidance during the session **[20__ – 20__]**, in partial fulfilment of the requirements for the award of the degree of Bachelor of Computer Applications of **[Name of University]**.

To the best of my knowledge, the work reported here meets the standard required for submission.

(Signature)

[Name of Project Guide]

Project Guide

(Signature)

[Name of HOD]

HOD, BCA Deptt.

(Signature)

External Examiner

Date: **[.....]**

ACKNOWLEDGEMENT

The completion of this project would not have been possible without the support and guidance of many people, and I take this opportunity to thank all of them.

I express my sincere gratitude to my project guide, **[Name of Project Guide]**, for the valuable guidance, constant encouragement and constructive suggestions given to me at every stage of this project.

I am thankful to **[Name of HOD]**, Head of the BCA Department, and to all the faculty members of the department for providing the facilities and the academic environment that made this work possible.

I also thank **[Name of Principal / Director]** and the management of **[Name of College]** for giving me the opportunity to undertake this project.

Finally, I am grateful to my parents, family and friends for their patience, cooperation and motivation throughout the course of this work. Their feedback while testing the game helped me improve it.

[Student Name]

BCA 5th Sem

Roll No: **[.....]**

ABSTRACT

MindLab – Science Memory Challenge is a web-based game that joins two activities that are usually kept apart: training short-term memory and learning science. The game is built on the familiar card-matching idea, in which cards are placed face down and the player has to remember their positions to find matching pairs. Instead of ordinary pictures, every card in MindLab shows a science element such as DNA, an atom, a flask or a planet. Each time a pair is matched, the game shows a short fact about that element, so the player learns something with every correct move.

The player chooses one of four science categories (Biology, Physics, Chemistry and Space) and one of three difficulty levels (Easy, Medium and Hard). During a round the application records the time taken, the number of attempts and the accuracy. At the end of the round it produces a performance report with a score, an accuracy figure, the completion time and a five-level memory rating, and it stores the result in a leaderboard.

The application is developed as a single-page application using React and JavaScript, with Vite as the build tool. It runs completely in the web browser and needs no server or database: scores and the player name are stored on the device using the browser's local storage. The interface is responsive and adapts to desktop, tablet and mobile screens. The system was tested with 42 function-level checks and 43 system-level checks, all of which passed.

Keywords: memory game, science education, educational game, React, JavaScript, single-page application, local storage, responsive web design.

INTRODUCTION OF THE PROJECT

5.1 Overview

MindLab – Science Memory Challenge is an interactive web application that combines a memory-training game with science learning. The player is shown a board of face-down cards. Two cards are turned over at a time, and the aim is to find all the matching pairs in as few attempts and as little time as possible.

What makes MindLab different from a normal memory game is its content. The cards carry science elements, grouped into four categories: Biology, Physics, Chemistry and Space. When the player matches two cards, a popup appears with a short and interesting fact about that element. For example, matching the two Atom cards shows that an atom is the smallest unit of an element, with a nucleus of protons and neutrons surrounded by electrons.

The application opens with a splash screen and then a home page. From there the player enters a name, selects a category and a difficulty level, and plays a round. After the round, a performance report is displayed and the score is saved to a leaderboard.

5.2 Need for the Project

Card-matching games are a simple and well-known way to exercise concentration and short-term visual memory. Most such games, however, use pictures of fruits, animals or shapes, and the player gains nothing from them beyond the game itself. On the other hand, science is often learned by reading and memorising text, which many students find dull.

This project brings the two together. Because the player must look closely at each science card to remember where it is, and is rewarded with a fact after each match, the game repeats science vocabulary and ideas in an enjoyable way. The project was therefore taken up to build something that is more than an ordinary game: a small platform for memory training, gaming and science education together.

5.3 Existing System

The existing alternatives fall into two groups, and each has limitations:

- **Ordinary memory games** use unrelated pictures. They provide entertainment but no educational content, and usually report only whether the player has won.
- **Science learning applications and websites** are mostly based on reading text, watching videos or answering quizzes. They do not train memory and are less engaging for casual use.
- Many games of both kinds need the user to install an application or create an account before playing.
- Few of them give the player a clear report of performance, such as accuracy and time, that can be compared from one round to the next.

5.4 Proposed System

The proposed system, MindLab, overcomes these limitations with the following features:

- A card-matching game whose cards are science elements, in four categories with 15 elements each (60 elements in total).
- A science fact shown in a popup after every successful match, and a running list of the facts discovered in the round.
- Three difficulty levels: Easy (6 pairs), Medium (8 pairs) and Hard (15 pairs), each with its own preview time and score multiplier.
- A short preview at the start of each round, in which all cards are shown so that the player can memorise them.
- Live statistics during play: timer, pairs found, attempts and accuracy.
- A performance report after each round showing score, accuracy, completion time, mistakes and a five-level memory rating.
- A leaderboard that stores scores on the device and can be filtered by category and difficulty.
- A responsive interface that works on desktop computers, tablets and mobile phones, with nothing to install and no login.

5.5 Scope of the Project

The project covers the complete design and development of the game as a client-side web application. It includes the user interface, the game logic, the scoring system, the science content and the storage of scores on the device. It does not include a server, user accounts or an online leaderboard shared between devices; these are described as future scope in Chapter 13.

5.6 Technology Used

The technologies used to build the project are summarised in Table 5.1.

Table 5.1: Technologies used in the project		
Area	Technology	Purpose in this project
Programming language	JavaScript (ES modules), JSX	Game logic and user interface code
User interface library	React 19	Building the screens from reusable components and managing state
Build tool	Vite 8	Development server with hot reload and the production build
Styling	CSS3 (custom properties, Grid, Flexbox, container queries)	Design system, layout and responsive behaviour
Icons	Lucide (lucide-react)	Science icons shown on the cards and interface icons
Data storage	Web Storage API (localStorage)	Saving the player name and the leaderboard on the device
Code quality	Oxlint	Static analysis of the source code
Version control	Git and GitHub	Source code management

OBJECTIVES OF THE PROJECT

The main objective of the project is to develop an interactive application that combines memory training, gaming and science education in one platform. The specific objectives are listed below.

1. To design and develop a card-matching memory game that runs in any modern web browser without installation.
2. To use science elements from Biology, Physics, Chemistry and Space as the content of the cards instead of ordinary pictures.
3. To display a short, accurate and interesting science fact every time the player matches a pair, so that learning happens during play.
4. To provide three difficulty levels so that the game suits both beginners and experienced players.
5. To measure the performance of the player during a round using a timer, the number of attempts and the accuracy.
6. To generate a performance report at the end of each round showing the score, accuracy, completion time and a memory rating.
7. To maintain a leaderboard of previous scores so that players can track their improvement.
8. To build a clean, professional and responsive user interface that works well on desktop, tablet and mobile screens.
9. To write modular and maintainable code in which the science content can be extended easily.

6.1 Expected Outcome

The expected outcome is a working application in which a player can complete the full flow, from entering a name to viewing the leaderboard, on any common device. This has been achieved, as shown in Chapters 10 and 12.

HARDWARE AND SOFTWARE REQUIREMENTS

7.1 Hardware Requirements

The hardware needed to develop the project and the hardware needed to use it are listed in Tables 7.1 and 7.2. The requirements are small because the application runs entirely in a web browser.

Table 7.1: Hardware requirements for development

Component	Minimum	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5 / Apple M-series or higher
RAM	4 GB	8 GB or more
Free disk space	1 GB (Node.js, project packages and the code editor)	2 GB or more
Display	1366 × 768	1920 × 1080
Input devices	Keyboard and mouse	Keyboard and mouse
Network	Internet connection to install packages	Broadband connection

Table 7.2: Hardware requirements for the user

Component	Requirement
Device	Any desktop computer, laptop, tablet or smartphone that can run a modern web browser
Screen	Minimum width of 320 pixels. The layout adapts automatically to larger screens
Input	Mouse, touchpad or touch screen
Storage	A few kilobytes of browser storage for the player name and scores

7.2 Software Requirements

Table 7.3 lists the software used during development with the versions installed, and Table 7.4 lists what the user needs.

Table 7.3: Software used for development		
Software	Version used	Purpose
Operating system	Windows 10/11, macOS or Linux	Development platform
Node.js	22.18.0 (Vite 8 needs 20.19 or later, or 22.12 or later)	JavaScript runtime for the build tools
npm	10.9.3	Installing and managing packages
React and React DOM	19.3.0	User interface library
Vite	8.3.3	Development server and production build
lucide-react	1.52.0	Icon set
Oxlint	1.87.0	Static code analysis
Code editor	Visual Studio Code	Writing and editing the source code
Web browser	Google Chrome 154	Running and testing the application
Git	Any recent version	Version control

Table 7.4: Software requirements for the user	
Software	Requirement
Web browser	A current version of Google Chrome, Microsoft Edge, Mozilla Firefox or Safari. The style sheet uses CSS container queries and the color-mix() function, which need Chrome or Edge 111, Firefox 113, Safari 16.2 or later
JavaScript	Must be enabled in the browser
Local storage	Must be enabled so that the player name and scores can be saved
Internet	Needed to open the hosted application and to load the web fonts. No other online service is used

PROJECT ANALYSIS & DESIGNING

8.1 System Analysis

System analysis is the study of the problem and of what the new system must do. The problem addressed by this project is that memory games give no learning value, while science learning tools give little memory practice and are not very engaging. The analysis below lists what the system must do (functional requirements) and how well it must do it (non-functional requirements).

Table 8.1: Functional requirements

No.	Requirement
FR1	The system shows a splash screen when it opens and then moves to the home page.
FR2	The player can enter a name of up to 18 characters. If the name is left blank, the name "Player" is used.
FR3	The player can choose one of four science categories: Biology, Physics, Chemistry or Space.
FR4	The player can choose one of three difficulty levels: Easy, Medium or Hard.
FR5	The system builds a shuffled deck in which every selected element appears exactly twice.
FR6	The system shows all cards for a short preview time before the round starts.
FR7	The player flips two cards in each attempt. A matching pair stays face up. A non-matching pair is turned back after 0.9 seconds.
FR8	After every match, the system shows a popup with a fact about the matched element.
FR9	During the round, the system displays the time, pairs found, attempts and accuracy.
FR10	At the end of the round, the system calculates a score and shows a performance report.
FR11	The system saves every result and shows a leaderboard of the ten best rounds, with filters for category and difficulty.
FR12	The player can restart a round, play again, change the mode and clear the saved scores.

Table 8.2: Non-functional requirements

Quality	Requirement
Usability	A new player should be able to start a round within three clicks and without reading instructions. A "How to play" page is also provided.
Performance	All processing happens in the browser. No network request is made during a round, so the game responds immediately.

Table 8.2: Non-functional requirements	
Quality	Requirement
Portability	The same application works on desktop, tablet and mobile screens through a responsive layout.
Reliability	If browser storage is unavailable or its content is damaged, the application keeps working and treats the leaderboard as empty.
Maintainability	Science content is kept in one data file, separate from the logic, so new elements can be added without changing the code.
Accessibility	Buttons and cards have text labels, keyboard focus is visible, and animations are reduced when the device requests reduced motion.

8.2 Feasibility Study

Technical feasibility. The project uses JavaScript, React and CSS, which are open, well-documented web technologies. They run in every modern browser, and the development tools (Node.js, Vite and a code editor) are freely available. No special hardware is needed. The project is therefore technically feasible.

Economic feasibility. All the software used is free and open source. Because the application has no server or database, there is no running cost. The built application consists of static files that can be hosted on any static web hosting service. The project is therefore economically feasible.

Operational feasibility. The application needs no installation, registration or training. A player only has to open it in a browser. The interface follows the familiar pattern of a card game, so it is easy to operate for students of any age group.

8.3 Development Model

The project was developed using the **iterative and incremental model**. A small working version was built first and then improved in steps. Each step added one group of features and was tested before the next step began. This model was chosen because the visual design and the layout needed several rounds of feedback and refinement.

Table 8.3: Increments of development	
Increment	Work done
1. Core game	Science content, deck building and shuffling, card flipping, pair matching and the fact popup.
2. Application flow	Splash screen, home page, category and difficulty selection, and navigation between screens.
3. Performance tracking	Timer, attempts, accuracy, scoring formula, performance report and leaderboard with local storage.
4. Visual design	Design system with typography, tactile buttons, hairline surfaces and vector icons.
5. Responsive layout	Full-width desktop layout with navigation, board-and-sidebar game screen, tablet and phone layouts.
6. Colour and testing	One colour for each science field applied across the interface, followed by unit and system testing.

8.4 System Design

MindLab is a single-page application. The whole program is loaded once, and the screens are changed by the program itself without loading new pages. The design is divided into three layers, shown in Table 8.4. Each layer depends only on the layers below it.

Table 8.4: Layers of the application		
Layer	Files	Responsibility
Presentation	<code>src/screens</code> , <code>src/components</code> , <code>src/styles</code>	Draws the screens, receives clicks and shows results. Screens are built from small reusable components such as Card, FactPopup and StatsBar.
Logic	<code>src/utils/game.js</code> , <code>src/hooks</code> , <code>src/App.jsx</code>	Builds and shuffles the deck, checks matches, runs the timer, calculates accuracy, score and rating, and controls which screen is shown.
Data	<code>src/data/science.js</code> , <code>src/utils/storage.js</code>	Holds the science content and difficulty settings, and reads and writes the player name and scores in the browser's local storage.

The movement of the player between screens is shown in Figure 8.1. The screen to display is held in one state variable in `App.jsx`, which works as a small state machine.

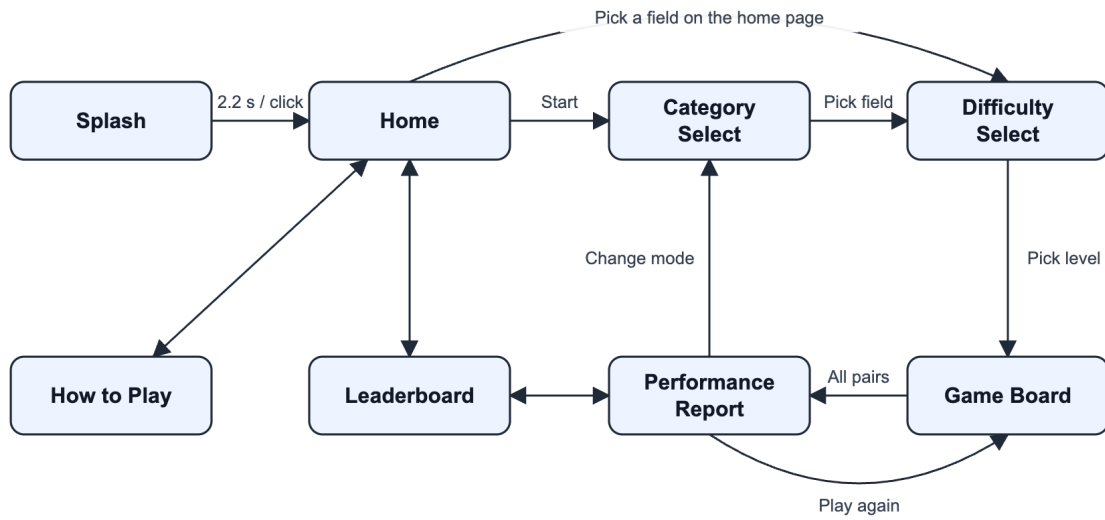


Figure 8.1: Screen flow of the application

8.5 Data Flow Diagrams (DFD)

A Data Flow Diagram shows how data moves through a system: where it comes from, which processes change it, where it is stored and where it goes. The symbols used in the diagrams of this report are explained in Table 8.5.

Table 8.5: DFD symbols used		
Symbol	Name	Meaning
Rectangle	External entity	A person or system outside the application that gives data to it or receives data from it. Here it is the Player.
Circle	Process	An activity that receives data, works on it and produces output. Each process has a number.
Open-ended box	Data store	A place where data is kept. Each data store has an identifier such as D1.
Arrow	Data flow	The movement of data. The label on the arrow names the data.
Dashed box	Linked process	A process from another diagram, shown only to mark where data comes from or goes to.

8.5.1 Level 0 DFD (Context Diagram)

The context diagram in Figure 8.2 shows the whole system as a single process. The Player supplies a name, a choice of category and level, and card selections. The system returns the game board, science facts, live statistics, the performance report and the leaderboard. The browser's local storage is shown as an external entity because it belongs to the browser and not to the application.

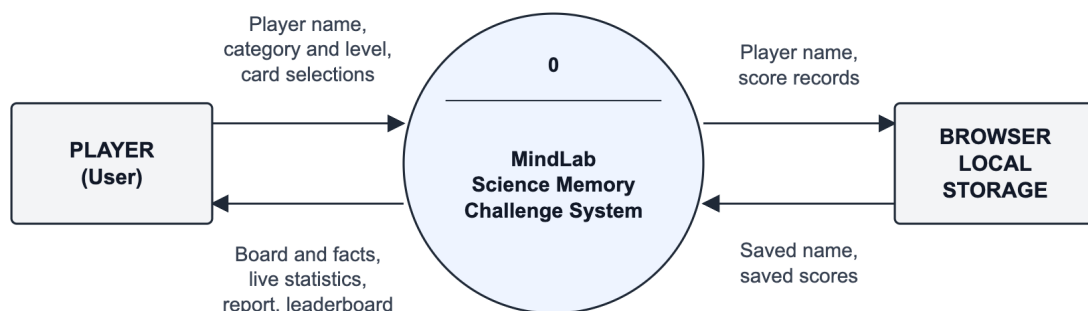


Figure 8.2: Level 0 DFD (context diagram)

8.5.2 Level 1 DFD

The Level 1 diagram in Figure 8.3 breaks the system into its six main processes and shows the data stores they use. The processes are described in Table 8.6. The data store D1 is drawn twice only to avoid crossing lines.

Table 8.6: Processes of the Level 1 DFD

No.	Process	Description
1.0	Manage Player Profile	Accepts the player name and saves it in D1 so that it is remembered on the next visit.
2.0	Select Category and Difficulty	Shows the categories and levels from D2 and passes the player's choice to process 3.0.
3.0	Generate Game Board	Picks the required number of elements from D2, duplicates them and shuffles them into a deck.
4.0	Play Round	Handles the preview, card flips, matching, fact popups and live statistics. It is expanded in the Level 2 DFD.
5.0	Compute Score and Report	Calculates the score, accuracy and memory rating, shows the report and writes a score record to D3.
6.0	Maintain Leaderboard	Reads the score records from D3, applies the filters chosen by the player and shows the ten best.

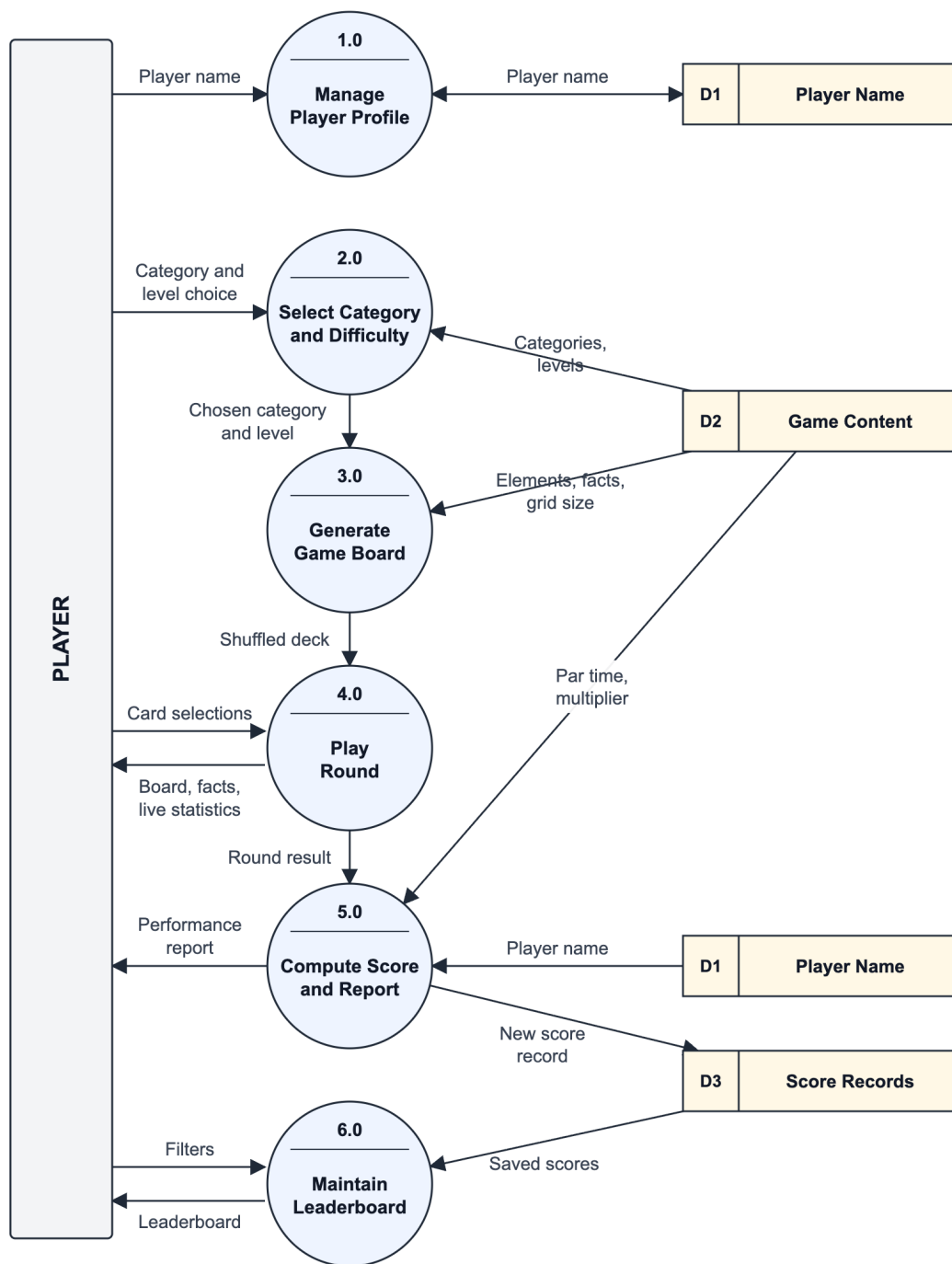


Figure 8.3: Level 1 DFD

8.5.3 Level 2 DFD of Process 4.0 (Play Round)

Figure 8.4 expands process 4.0. The round begins when the shuffled deck arrives from process 3.0. The cards are revealed briefly (4.1). Each card selected by the player is flipped (4.2), and when two cards are face up they are compared (4.3). The result of the comparison is recorded in the round state (D4), which

holds the flipped cards, the matched cards, the number of attempts and the time. A matched element is passed to process 4.4, which shows its fact. Process 4.5 keeps the timer and statistics up to date, and process 4.6 checks whether every pair has been found and, if so, sends the round result to process 5.0.

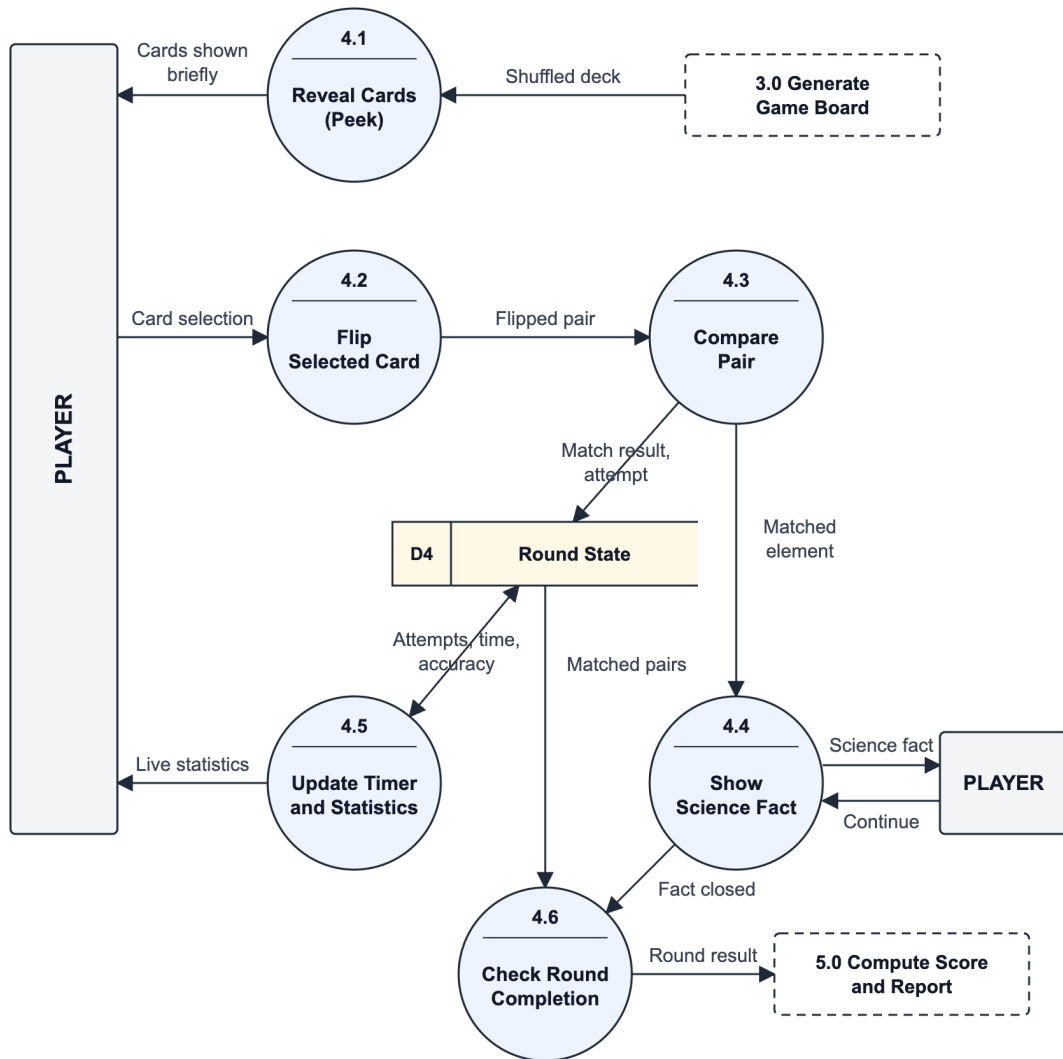


Figure 8.4: Level 2 DFD of process 4.0 (Play Round)

8.6 Entity Relationship (ER) Diagram

An Entity Relationship diagram shows the things about which data is kept (entities), their properties (attributes) and how they are related. The ER diagram of MindLab is given in Figure 8.5. It has five entities.

- **PLAYER** is the person who plays. A player is identified by the player name.
- **SCORE** is the result of one completed round.
- **CATEGORY** is a field of science, such as Biology.
- **ELEMENT** is one science item shown on a pair of cards, together with its fact.
- **DIFFICULTY** is a level of play with its settings, such as the number of pairs.

Table 8.7: Relationships between the entities

Relationship	Between	Type	Meaning
achieves	PLAYER and SCORE	One to many	One player can have many scores. Each score belongs to one player.
is played in	SCORE and CATEGORY	Many to one	Many scores can be recorded in one category. Each score is for one category.
is played at	SCORE and DIFFICULTY	Many to one	Many scores can be recorded at one level. Each score is for one level.
contains	CATEGORY and ELEMENT	One to many	One category contains many elements (15 each). Each element belongs to one category.

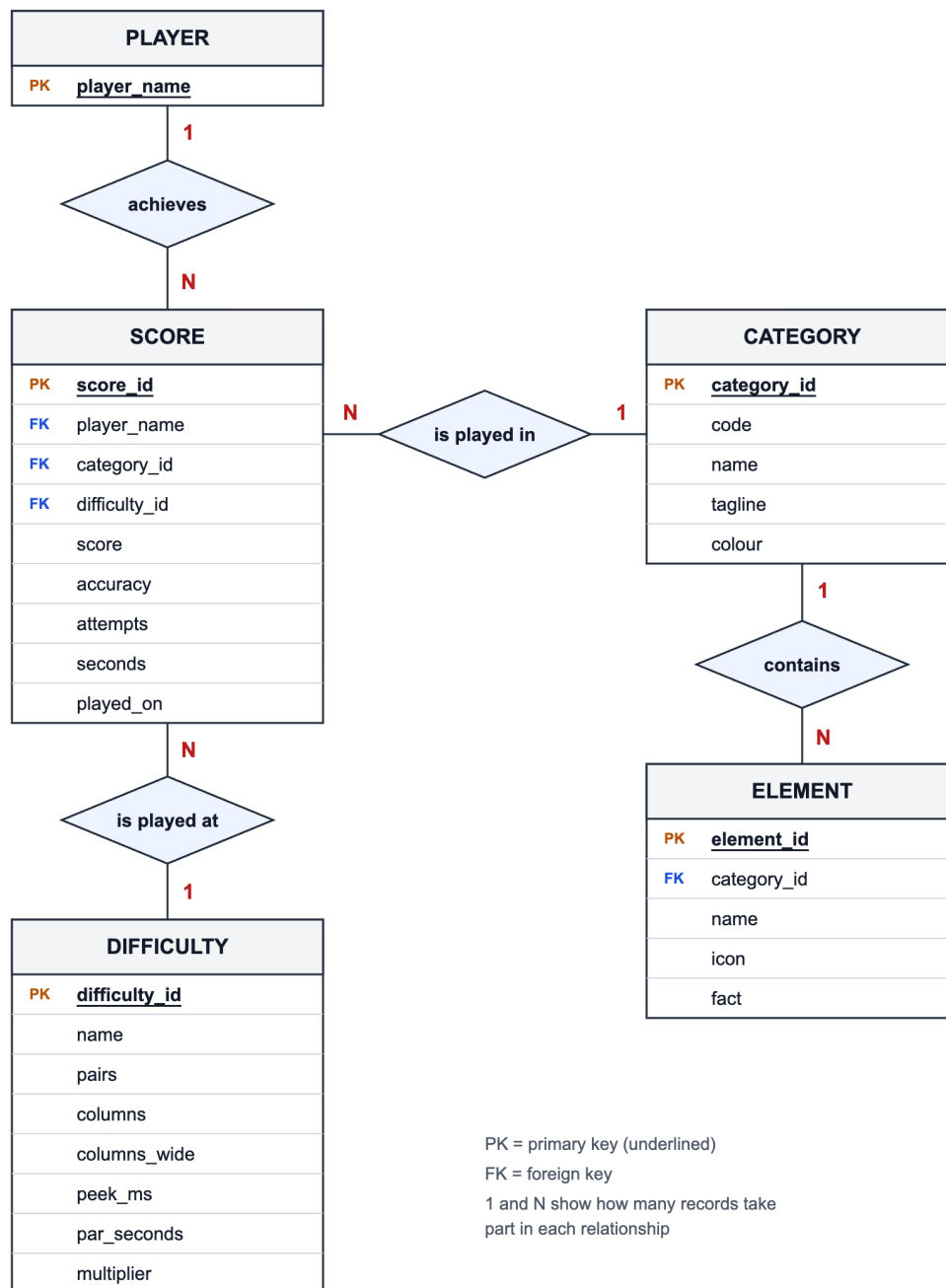


Figure 8.5: ER diagram of MindLab

Note on storage. MindLab does not use a database server. The ER diagram describes the logical structure of the data, and Table 8.8 shows where each entity is physically kept. If the project is later extended with an online leaderboard, the same diagram can be turned directly into database tables.

Table 8.8: Physical storage of each entity

Entity	Where it is stored
PLAYER	Browser local storage, under the key <code>mindlab.playerName</code> .
SCORE	Browser local storage, under the key <code>mindlab.scores</code> , as a JSON list sorted from the highest score to the lowest. Only the best 50 records are kept.
CATEGORY, ELEMENT, DIFFICULTY	Constant data in the file <code>src/data/science.js</code> , which is part of the program. This data is read-only for the player.

8.7 Data Dictionary

The data dictionary lists every field of each entity with its type and meaning.

Table 8.9: SCORE record

Field	Type	Example	Description
id	String	1791543478729-9py5e	Unique identifier made from the time of saving and a random suffix (primary key).
name	String (max 18)	Indu	Name of the player who played the round.
category	String	physics	Identifier of the category played.
difficulty	String	medium	Identifier of the level played.
score	Integer	1416	Points earned in the round.
accuracy	Integer (0–100)	89	Percentage of attempts that produced a match.
attempts	Integer	9	Number of times two cards were compared.
seconds	Integer	22	Time taken from the first flip to the last match.
date	String (ISO 8601)	2026-10-09T10:57:58.729Z	Date and time when the round was completed.

Table 8.10: CATEGORY

Field	Type	Example	Description
id	String	biology	Unique identifier of the category (primary key).
code	String (3)	BIO	Short code shown on the fact popup.
name	String	Biology	Name shown to the player.
tagline	String	Cells, organs & living things	One-line description.

Table 8.10: CATEGORY			
Field	Type	Example	Description
icon	Icon component	Dna	Icon that represents the category.
color and four tints	String (hex colour)	#059669	Colour of the category and its lighter and darker steps, used to theme the screens.

Table 8.11: ELEMENT			
Field	Type	Example	Description
id	String	atom	Identifier, unique inside its category (primary key).
name	String	Atom	Name printed on the card.
icon	Icon component	Atom	Picture shown on the face of the card.
fact	String	An atom is the smallest unit of an element...	Fact shown in the popup when the pair is matched.

Table 8.12: DIFFICULTY and its three records				
Field	Description	Easy	Medium	Hard
pairs	Number of pairs to find	6	8	15
columns	Columns of the board on a phone	4	4	5
columnsWide	Columns of the board on a desktop	4	4	6
peekMs	Preview time in milliseconds	3000	2500	2000
parSeconds	Target time in seconds, used for the time bonus	45	75	150
multiplier	Number by which the score is multiplied	1	1.5	2

8.8 User Interface Design

The interface was designed to look clean and professional, and to keep the attention of the player on the cards. The following principles were followed.

- **Colour has a meaning.** Each science field has one flat colour, and that colour is used for its icon, its card backs and the screens of a round played in it. No gradients are used.

- **Neutral frame.** Navigation, text and primary buttons are black, white and grey, so that the field colours stand out.
- **Clear typography.** A serif typeface is used for headings, a sans-serif typeface for text and a monospaced typeface for small labels.
- **Immediate feedback.** Cards flip with a short animation, matched cards are tinted and ticked, and the statistics change at once.
- **Responsive layout.** The layout changes with the width of the screen, as shown in Table 8.14.

Table 8.13: Colour palette

Use	Colour	Hex code
Biology, and “success” states	Emerald	#059669
Physics, and information	Blue	#2563EB
Chemistry, and “Medium” level	Amber	#D97706
Space, and “Hard” level or mistakes	Rose	#E11D48
Text, navigation and primary buttons	Ink	#171717
Page background and cards	White	#FFFFFF

Table 8.14: Layout at different screen widths

Screen width	Layout
1000 pixels and above (desktop)	Top navigation with text links, two-column home page, four category cards in a row, and a game screen with the board on the left and statistics on the right. The Hard board has 6 columns.
620 to 999 pixels (tablet)	Sections are stacked, card grids have two or three columns, and the board is centred.
Below 620 pixels (phone)	Navigation shows icons only, lists become compact rows and the Hard board has 5 columns of square cards so that the whole board stays in view.

MODULES

9.1 List of Modules

The application is divided into nine modules. Each module has one clear responsibility and is implemented in a small number of files, as shown in Table 9.1.

Table 9.1: Modules of the application			
No.	Module	Main files	Responsibility
1	Application shell and navigation	App.jsx, Shell.jsx, Splash.jsx	Shows the splash screen, the navigation bar and the correct screen.
2	Home and player profile	Home.jsx, storage.js	Takes the player name and shows the player's summary.
3	Category and difficulty selection	CategorySelect.jsx, DifficultySelect.jsx, CategoryCard.jsx	Lets the player choose what to play.
4	Game board	Game.jsx, Card.jsx, game.js	Builds the deck and runs the round.
5	Fact popup	FactPopup.jsx	Shows the fact for a matched element.
6	Timer and live statistics	useTimer.js, StatsBar.jsx	Measures time and shows the statistics.
7	Scoring and performance report	game.js, Report.jsx, ProgressRing.jsx	Calculates the score and shows the report.
8	Leaderboard and storage	Leaderboard.jsx, storage.js, Segmented.jsx	Saves, filters and shows the scores.
9	How to play	HowToPlay.jsx	Explains the rules in six steps.

9.2 Process Logic of Each Module

Module 1: Application Shell and Navigation

Purpose: to control which screen is visible. **Input:** clicks on navigation links and buttons. **Output:** the selected screen.

1. When the application opens, set the current screen to `splash`.
2. After 2.2 seconds, or when the player clicks, set the current screen to `home`.

3. For every other screen, draw the navigation bar and, below it, the component of the current screen.
4. When a navigation link or button is clicked, change the current screen. The possible screens are `home`, `howto`, `category`, `difficulty`, `game`, `report` and `leaderboard`.
5. Whenever the screen changes, scroll the page to the top.

Module 2: Home and Player Profile

Purpose: to welcome the player and take the name. **Input:** player name.

Output: saved name, and a summary of rounds played, best score and average accuracy.

1. Read the saved name and the saved scores from local storage.
2. Show the name in the input box. The box accepts at most 18 characters.
3. Calculate the summary: number of rounds, highest score and average accuracy of all saved rounds.
4. When the player presses “Start challenge” or picks a category card, remove extra spaces from the name. If the name is empty, use “Player”. Save the name in local storage.
5. Move to the category screen, or directly to the difficulty screen if a category card was picked.

Module 3: Category and Difficulty Selection

Purpose: to let the player choose a category and a level. **Input:** two clicks.

Output: the chosen category and difficulty, passed to the game board.

1. Show one card for each of the four categories with its icon, name, description, number of cards and the best score so far in that category.
2. When a category is clicked, remember it and show the three difficulty levels in the colour of that category.
3. For each level, show a small preview of the board, the number of pairs, the preview time, the target time and the score multiplier.
4. When a level is clicked, remember it and open the game board.

Module 4: Game Board

Purpose: to run one round of the game. **Input:** category, difficulty and the cards clicked by the player. **Output:** the result of the round (score, accuracy, attempts, time and the elements found). The logic is shown as a flowchart in Figure 9.1.

Building the deck. The deck is built by the function `buildDeck`:

1. Take the 15 elements of the chosen category and shuffle them.
2. Keep the first N elements, where N is the number of pairs for the chosen level.
3. Create two cards for each element, so the deck has 2N cards.
4. Shuffle the 2N cards.

Shuffling uses the **Fisher–Yates algorithm**, which gives every possible order the same chance: starting from the last position i and moving down to position 1, pick a random position j from 0 to i and exchange the cards at positions i and j.

Playing the round. The steps followed when the player clicks a card are:

1. Ignore the click if the preview is still running, if the board is locked, or if the card is already face up or matched.
2. If this is the first click of the round, start the timer.
3. Turn the card face up. If only one card is face up, wait for the next click.
4. When two cards are face up, add 1 to the number of attempts and lock the board.
5. If both cards show the same element, mark them as matched, add the element to the list of discoveries and open the fact popup. The board is unlocked when the popup is closed.
6. If the cards are different, wait 0.9 seconds, turn both face down and unlock the board.
7. When every pair is matched and the last popup is closed, stop the timer, calculate the score and pass the result to the report.

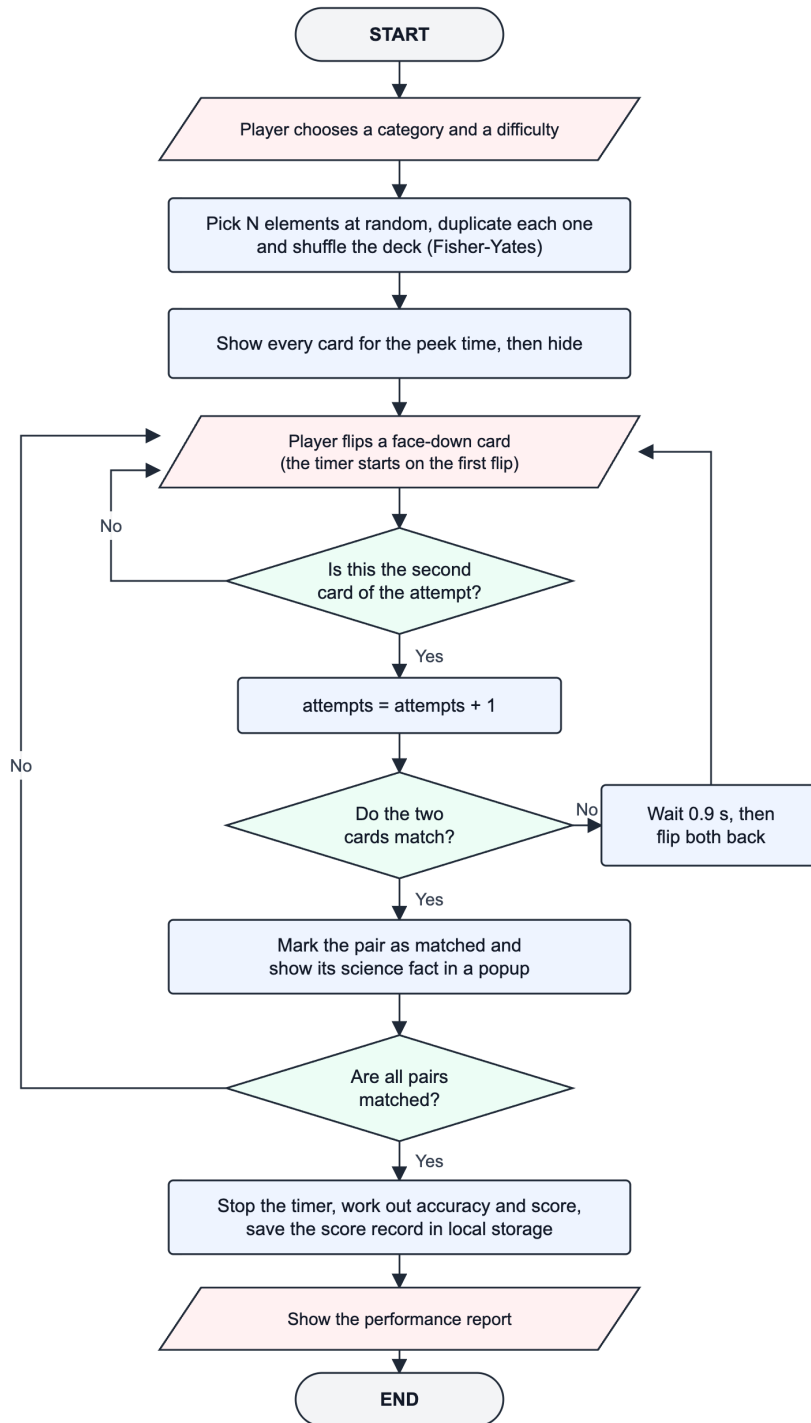


Figure 9.1: Flowchart of one round

Module 5: Fact Popup

Purpose: to teach a fact at the moment of success. **Input:** the matched element and its category. **Output:** a popup with the element's icon, name and fact.

1. Find the position of the element in its category and form a reference code, for example `PHY.05` for the fifth Physics element.

2. Show the code, the name of the element, the heading “Did you know”, the fact and the number of pairs found so far.
3. Keep the board locked while the popup is open.
4. Close the popup when the player presses “Continue”, presses the Escape key or clicks outside the popup.

Module 6: Timer and Live Statistics

Purpose: to measure and display the progress of the round. **Input:** the state of the round. **Output:** time, pairs found, attempts and accuracy.

1. When the first card is flipped, store the current clock time as the start time.
2. Four times every second, set the elapsed seconds to the whole number of seconds between the start time and the current time. Using the clock in this way keeps the timer correct even if the browser is slow.
3. Stop updating when the round is finished.
4. Calculate accuracy with the formula below and show the four statistics. A thin progress line shows the share of pairs found.

Accuracy (%) = (pairs matched ÷ attempts) × 100, rounded to a whole number. If there are no attempts, accuracy is 0.

Module 7: Scoring and Performance Report

Purpose: to turn the result of a round into a score and a report. **Input:** pairs, attempts, seconds and the difficulty settings. **Output:** score, accuracy, mistakes, memory rating and a recap of the facts.

The score rewards finding every pair, takes away points for wrong attempts, adds a bonus for finishing faster than the target time, and is multiplied according to the level:

$$\text{base} = \text{pairs} \times 100$$

$$\text{penalty} = (\text{attempts} - \text{pairs}) \times 15$$

$$\text{time bonus} = (\text{target time} - \text{seconds taken}) \times 3, \text{ or } 0 \text{ if the target time is exceeded}$$

$$\text{score} = (\text{base} - \text{penalty} + \text{time bonus}) \times \text{multiplier, rounded, and never less than } 0$$

Worked example. In the round shown in Figure 10.10, the player found 8 pairs at the Medium level in 9 attempts and 22 seconds. The target time for Medium is 75 seconds and the multiplier is 1.5.

- $\text{base} = 8 \times 100 = 800$
- $\text{penalty} = (9 - 8) \times 15 = 15$
- $\text{time bonus} = (75 - 22) \times 3 = 159$
- $\text{score} = (800 - 15 + 159) \times 1.5 = 944 \times 1.5 = \mathbf{1416}$
- $\text{accuracy} = 8 \div 9 \times 100 = \mathbf{89\%}$

The memory rating is decided from the accuracy as shown in Table 9.2.

Table 9.2: Memory performance rating			
Accuracy	Level	Rating	Message shown
90% and above	5	Exceptional	Near-perfect recall. Your memory is razor sharp.
75% to 89%	4	Sharp	Very few slips. You are tracking positions really well.
60% to 74%	3	Good	Solid memory with a few guesses along the way.
45% to 59%	2	Developing	You are getting there. Try to focus on card positions.
Below 45%	1	Warming up	Keep practising. Memory improves fast with repetition.

The report screen then carries out these steps:

1. Read the best earlier score for the same category and level.
2. Save the new score record (see Module 8).
3. Show the score and the accuracy ring, counting up from zero.
4. If the new score is higher than the best earlier score, show the “New best” badge.
5. Show the completion time, attempts, pairs matched and mistakes (attempts – pairs), the memory rating, and one card for each fact discovered in the round.

Module 8: Leaderboard and Storage

Purpose: to keep the scores and show the best ones. **Input:** a new score record, and the filters chosen by the player. **Output:** the list of the ten best rounds.

Saving a score:

1. Read the list of saved scores from local storage. If the stored text cannot be read, use an empty list.
2. Give the new record a unique identifier and add it to the list.
3. Sort the list from the highest score to the lowest. If two scores are equal, the one with the shorter time comes first.
4. Keep only the first 50 records and write the list back to local storage.

Showing the leaderboard:

1. Read the saved scores.
2. Keep only the records that match the selected category and the selected level. "All" keeps every record.
3. Show the first ten records with rank, player, field, level, accuracy, time, date and score.
4. If no record matches, show the message "No scores yet".
5. When "Clear scores" is pressed, ask for confirmation and then empty the list.

Module 9: How to Play

Purpose: to explain the game to a new player. The page shows six numbered steps (pick a category, choose a difficulty, memorise the preview, flip two cards, learn a fact and beat your score) and a button that starts a challenge.

INPUT / OUTPUT SCREENSHOT

This chapter shows the screens of the application as they appear in the browser. The screens on which the player gives input are shown first, followed by the screens that show output. All the screenshots were taken from the running application. The rounds shown were played by an automated test player, so the times are shorter than those of a human player.

10.1 Input Screens

Home page. The player types a name in the “Player name” box and presses “Start challenge”, or picks a field directly from the four category cards. The summary shows the rounds played, the best score and the average accuracy.

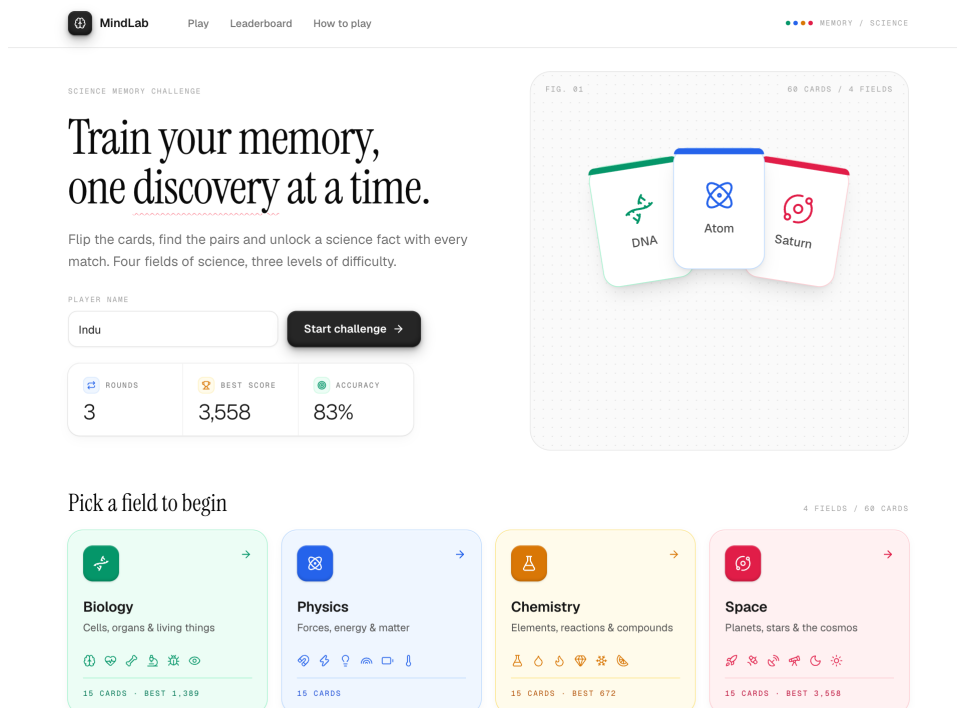


Figure 10.1: Home page with the player name input

Category selection. The player chooses one of the four science fields. Each card shows the number of cards and the best score in that field.

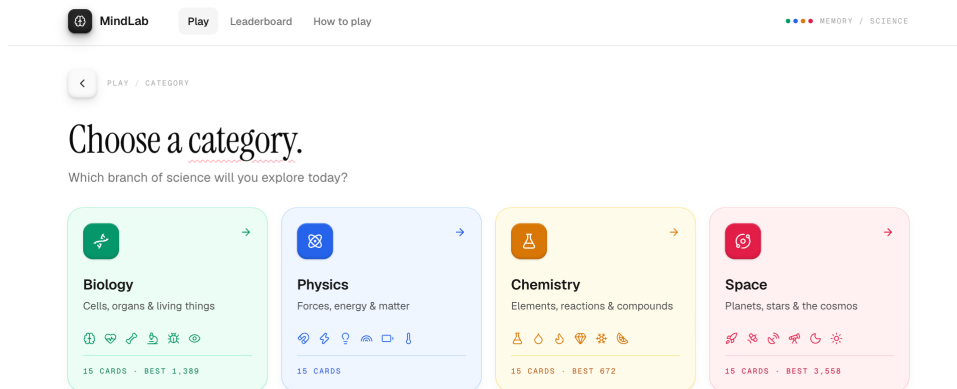


Figure 10.2: Category selection

Difficulty selection. The player chooses Easy, Medium or Hard. Each option shows a preview of the board, the number of pairs, the preview time and the target time.

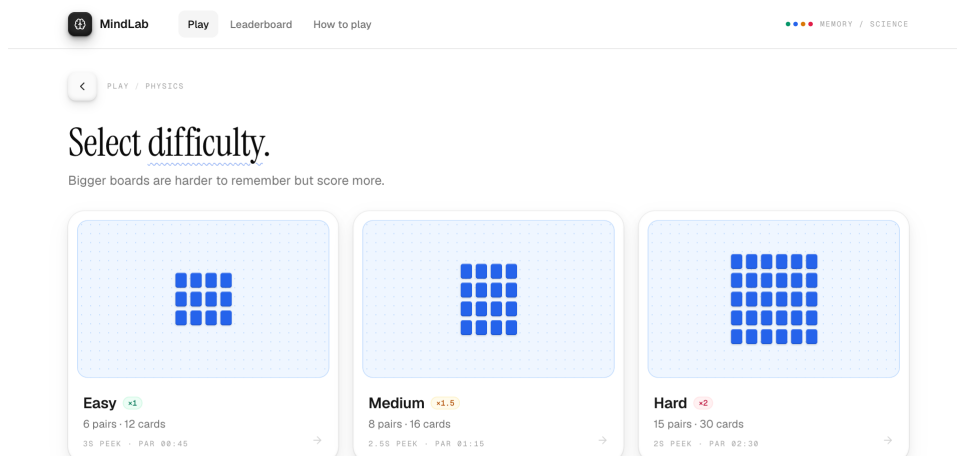


Figure 10.3: Difficulty selection for the Physics category

Card selection. After the preview, all cards are face down and the player selects cards by clicking them. The timer has not started yet.

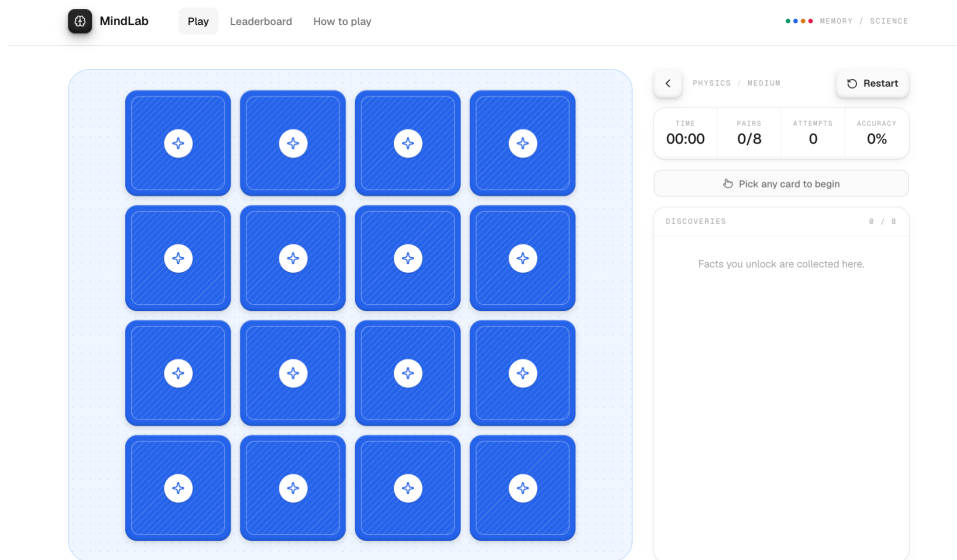


Figure 10.4: Board ready for card selection

10.2 Output Screens

Splash screen. This is the first screen shown when the application opens.



Figure 10.5: Splash screen

Preview. At the start of a round every card is shown for a few seconds. The bar under “Memorise the cards” shows the time left.

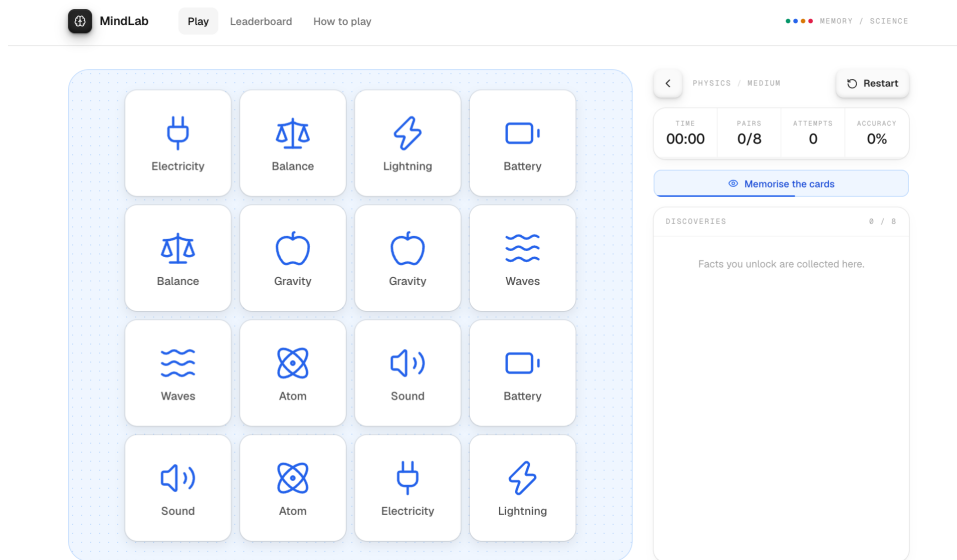


Figure 10.6: Preview of the cards at the start of a round

Round in progress. Matched pairs stay face up with a tick. The panel on the right shows the time, pairs, attempts and accuracy, and the facts discovered so far.

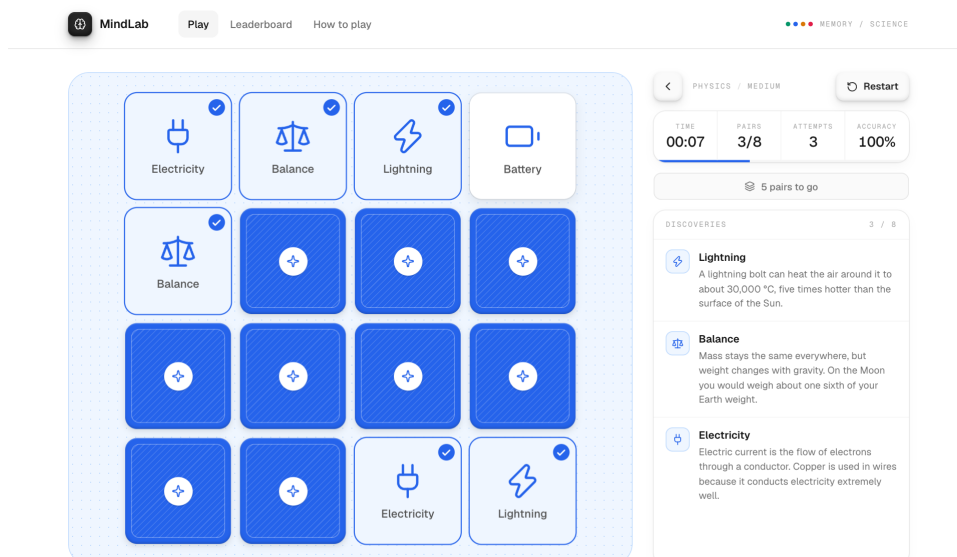


Figure 10.7: Round in progress with three pairs matched

Wrong attempt. Two different cards (Battery and Gravity) are face up. The attempt is counted, the accuracy falls to 75% and both cards turn back after 0.9 seconds.

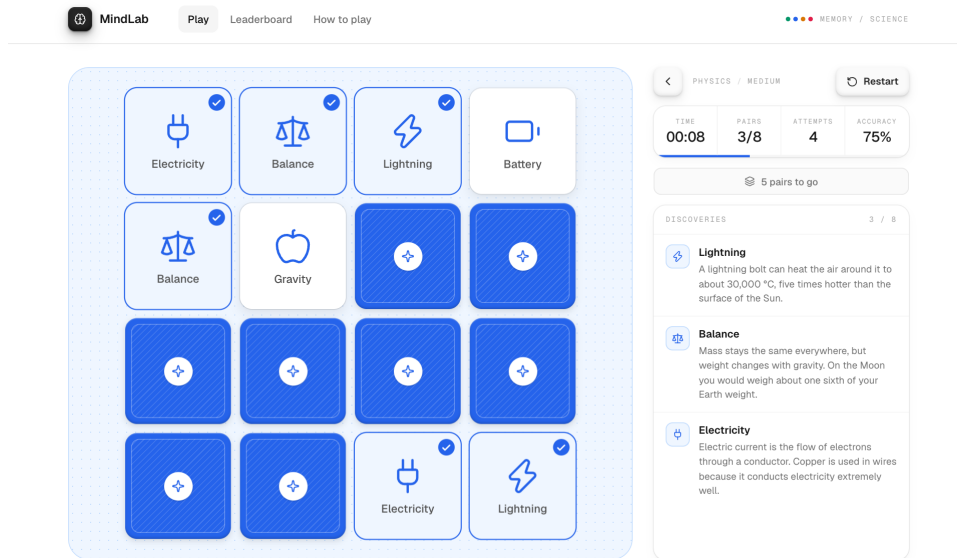


Figure 10.8: A wrong attempt

Fact popup. When a pair is matched, the popup shows the element, its reference code and a science fact.

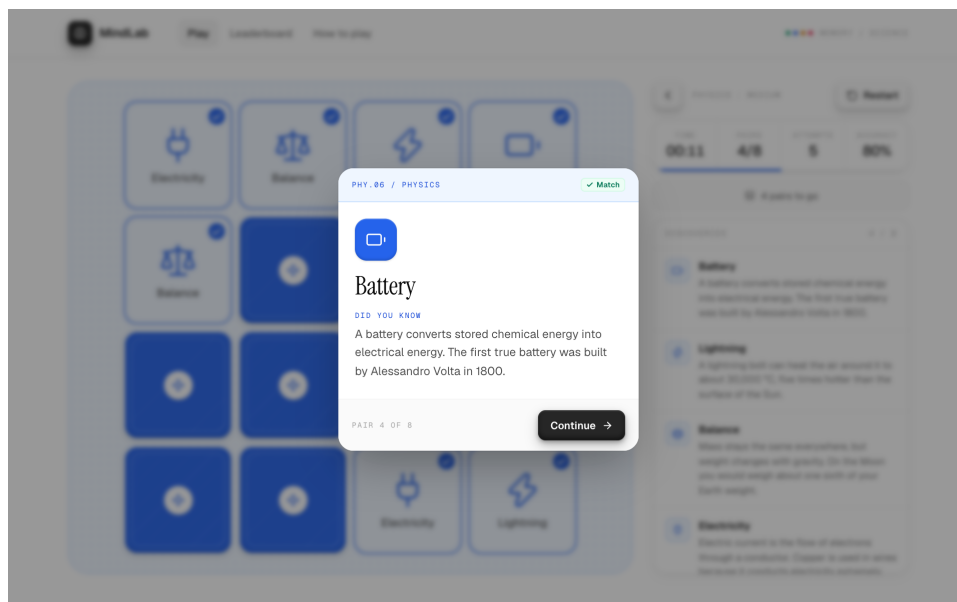


Figure 10.9: Fact popup after a successful match

Performance report. At the end of the round the report shows the score, accuracy, memory rating, completion time, attempts, pairs matched and mistakes, followed by all the facts discovered.

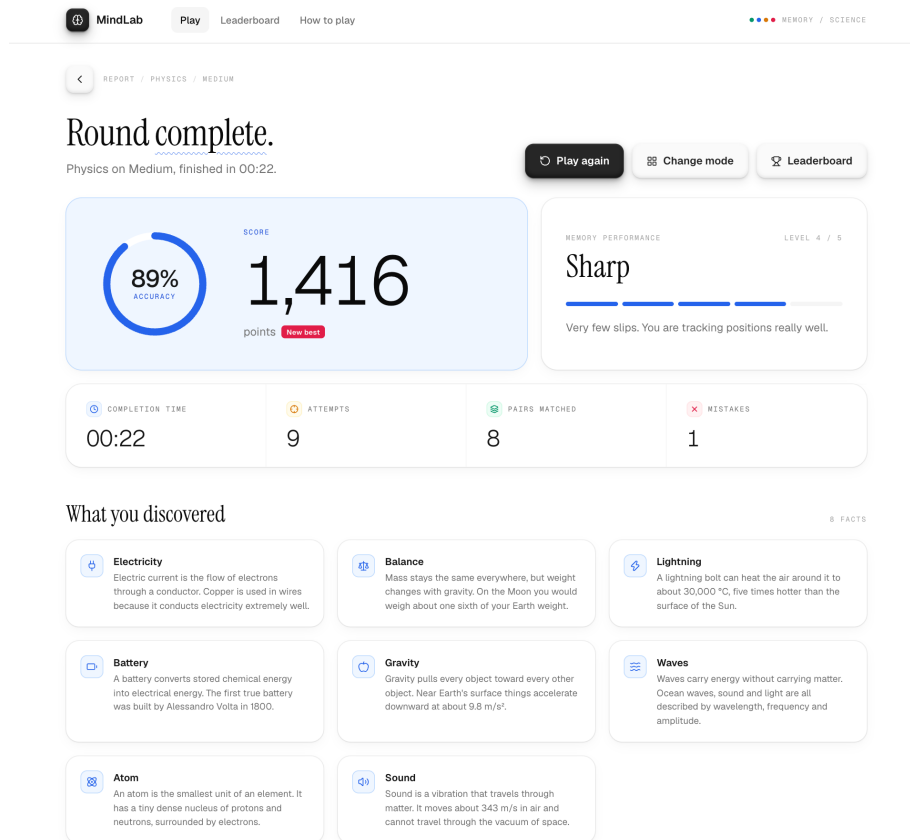


Figure 10.10: Performance report

Leaderboard. The saved rounds are listed from the highest score to the lowest.

The screenshot shows the MindLab application interface. At the top, there's a navigation bar with 'MindLab', 'Play', 'Leaderboard' (selected), and 'How to play'. On the right, it says 'MEMORY / SCIENCE'. Below the navigation bar, there's a 'SCORES / THIS DEVICE' section with a 'Clear scores' link. The main heading is 'Leaderboard.' followed by the subtitle 'The ten best rounds saved on this device.' Below this, there are two rows of filter buttons. The first row has 'All', 'Easy', 'Medium', and 'Hard'. The second row has 'All' (selected), 'Biology', 'Physics', 'Chemistry', and 'Space'. The main content is a table with 8 columns: #, PLAYER, FIELD, LEVEL, ACCURACY, TIME, DATE, and SCORE. It lists the top 4 rounds.

#	PLAYER	FIELD	LEVEL	ACCURACY	TIME	DATE	SCORE
1	Aman	Space	Hard	83%	00:42	09 Oct	3,558
2	Indu	Physics	Medium	89%	00:22	09 Oct	1,416
3	Indu	Biology	Medium	80%	00:23	09 Oct	1,389
4	Riya	Chemistry	Easy	86%	00:16	09 Oct	672

Figure 10.11: Leaderboard

Filtered leaderboard. Here the “Medium” filter is selected, so only the Medium rounds are listed.

This screenshot shows the same MindLab interface as Figure 10.11, but with the 'Medium' filter selected in the first row of buttons. Consequently, only two rounds are displayed in the table, both of which are at the 'Medium' difficulty level.

#	PLAYER	FIELD	LEVEL	ACCURACY	TIME	DATE	SCORE
1	Indu	Physics	Medium	89%	00:22	09 Oct	1,416
2	Indu	Biology	Medium	80%	00:23	09 Oct	1,389

Figure 10.12: Leaderboard filtered by difficulty

How to play. The rules of the game are explained in six steps.

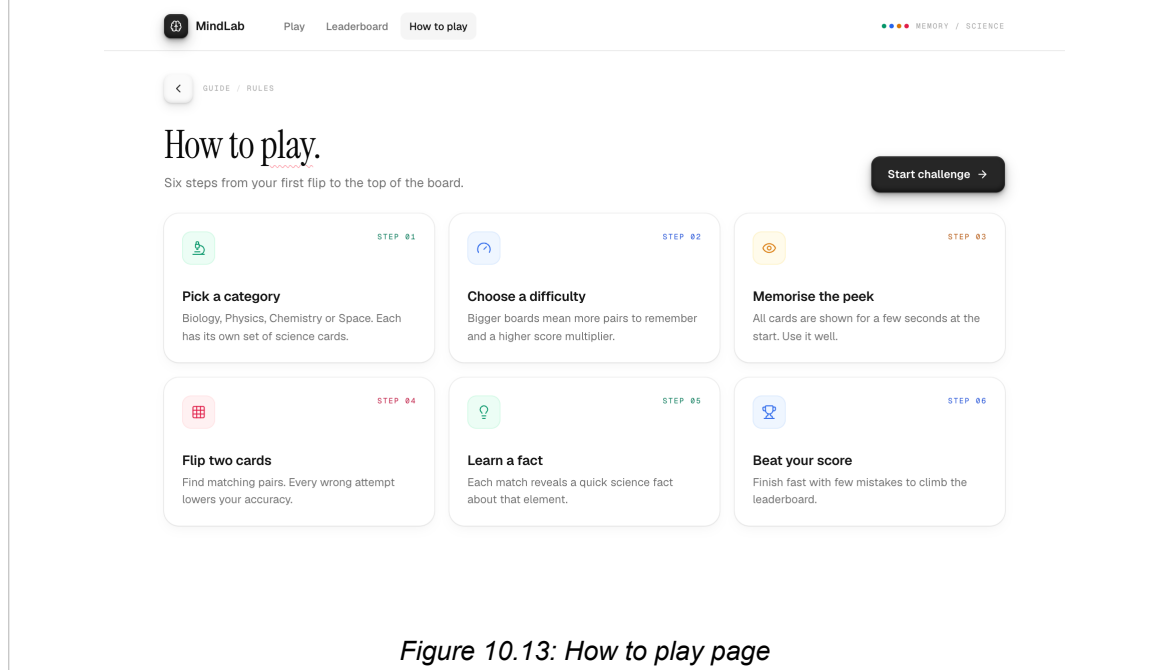


Figure 10.13: How to play page

10.3 Screens on a Mobile Phone

The same application adapts to a phone screen. Figures 10.14 and 10.15 show six screens at a width of 390 pixels.

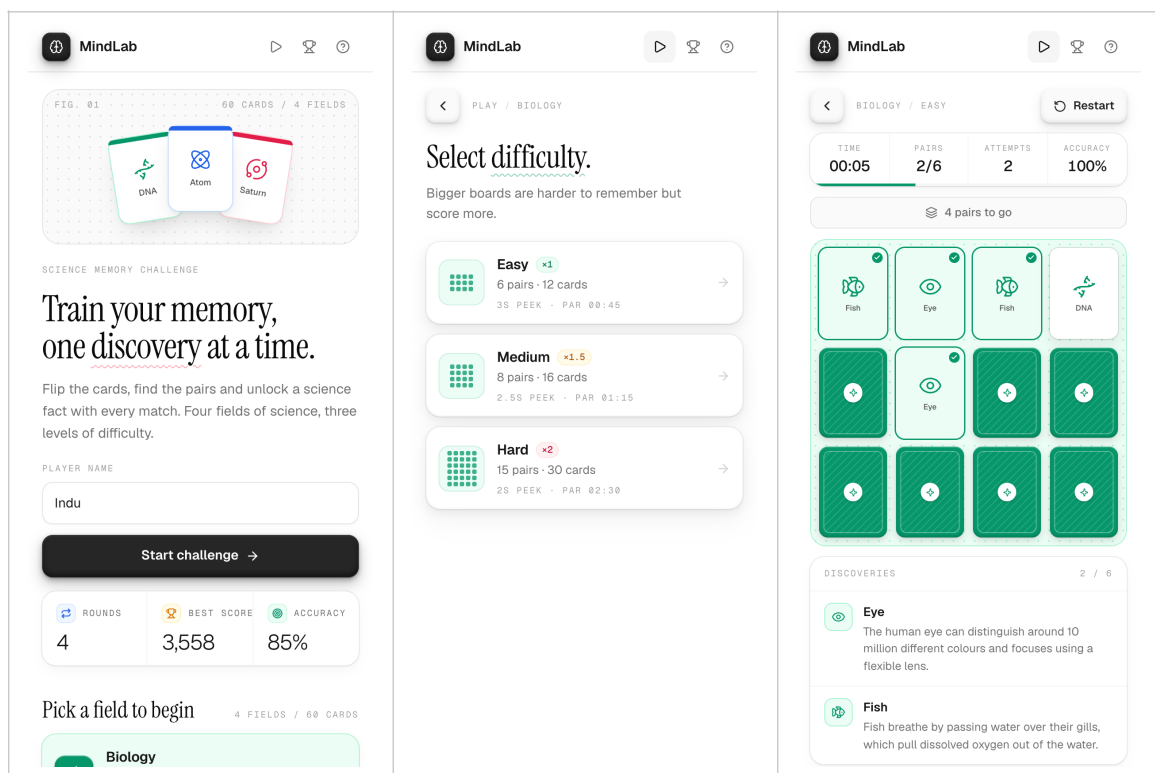


Figure 10.14: Home page, difficulty selection and game board on a phone

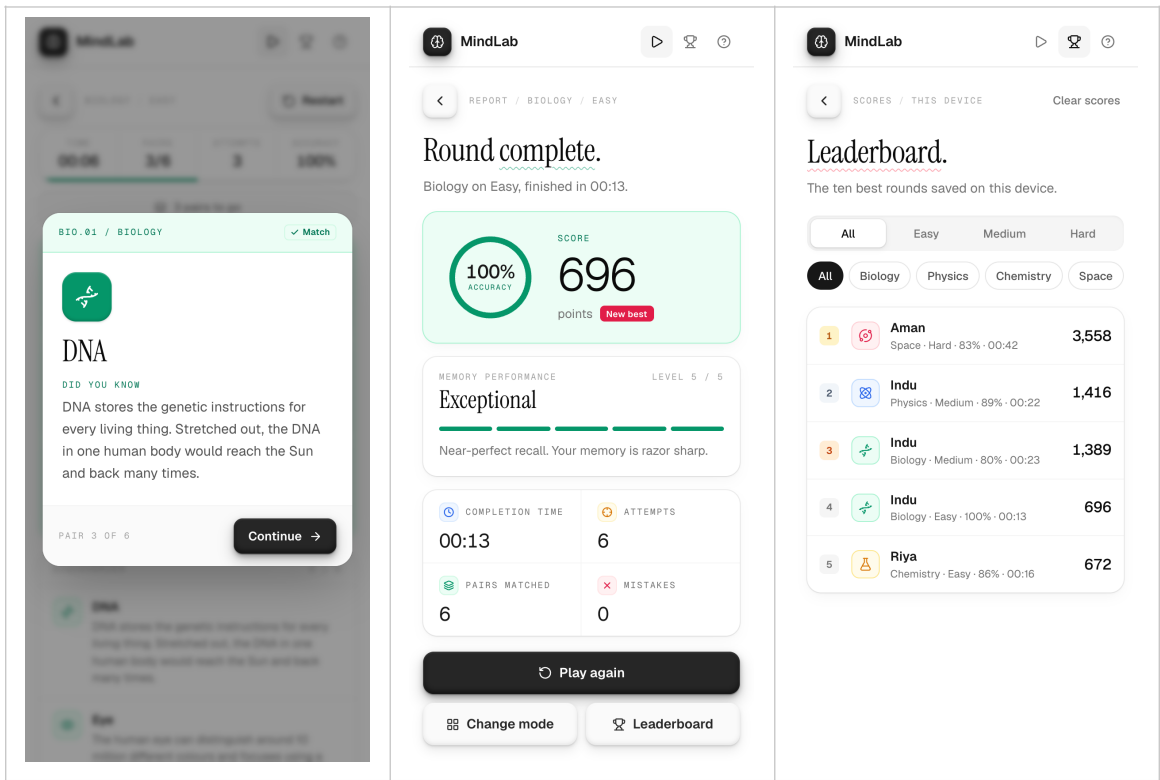


Figure 10.15: Fact popup, performance report and leaderboard on a phone

CODING

11.1 Structure of the Project

The source code is organised into folders according to the layers described in Section 8.4. The project contains about 2,750 lines of code in 25 source files, together with two test scripts.

Folder structure

mindlab/	
index.html	page that loads the application
package.json	project settings and list of packages
vite.config.js	build tool settings
src/	
main.jsx	starting point: mounts the App component
App.jsx	controls which screen is shown
data/	
science.js	categories, elements, facts, difficulty settings
utils/	
game.js	deck building, shuffle, accuracy, score, rating
storage.js	player name and scores in local storage
hooks/	
useTimer.js	round timer
useCountUp.js	count-up animation of numbers on the report
components/	
Shell.jsx	navigation bar and page frame
TopBar.jsx	back button and breadcrumb
PageHeader.jsx	page title and subtitle
CategoryCard.jsx	card for one science field
Card.jsx	one playing card
FactPopup.jsx	popup with the science fact
StatsBar.jsx	time, pairs, attempts, accuracy
ProgressRing.jsx	accuracy ring on the report
Segmented.jsx	difficulty filter on the leaderboard
screens/	
Splash.jsx	
Home.jsx	
HowToPlay.jsx	
CategorySelect.jsx	
DifficultySelect.jsx	
Game.jsx	
Report.jsx	
Leaderboard.jsx	
styles/	
index.css	design tokens and all styles
tests/	
unit.mjs	unit tests (npm test)
system.mjs	system tests in the browser (npm run test:system)

11.2 Running the Project

The following commands are used in the project folder. Node.js must be installed first.

Command	Purpose
<code>npm install</code>	Downloads the packages listed in package.json. This is needed only once.
<code>npm run dev</code>	Starts the development server. The application opens at <code>http://localhost:5173</code> .

Command	Purpose
<code>npm run build</code>	Creates the final, optimised files in the <code>dist</code> folder for hosting.
<code>npm run lint</code>	Checks the source code for mistakes using Oxlint.
<code>npm test</code>	Runs the unit tests.
<code>npm run test:system</code>	Runs the system tests in Google Chrome. The development server must be running.

11.3 Source Code

The important source files are listed below. The complete source code is maintained in a Git repository at <https://github.com/Indu0213/mind-lab>.

File: `src/main.jsx`

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './styles/index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

File: `src/App.jsx`

```
import { useCallback, useEffect, useState } from 'react'
import Shell from './components/Shell.jsx'
import Splash from './screens/Splash.jsx'
import Home from './screens/Home.jsx'
import HowToPlay from './screens/HowToPlay.jsx'
import CategorySelect from './screens/CategorySelect.jsx'
import DifficultySelect from './screens/DifficultySelect.jsx'
import Game from './screens/Game.jsx'
import Report from './screens/Report.jsx'
import Leaderboard from './screens/Leaderboard.jsx'
import { getBestScore, getPlayerName, saveScore } from './utils/storage.js'

// Which navigation item is highlighted on each screen.
const NAV_SECTION = {
  category: 'play',
  difficulty: 'play',
  game: 'play',
  report: 'play',
  leaderboard: 'leaderboard',
  howto: 'howto',
}

// Screens are driven by a small state machine rather than a router.
export default function App() {
  const [screen, setScreen] = useState('splash')
  const [categoryId, setCategoryId] = useState(null)
  const [difficultyId, setDifficultyId] = useState(null)
  const [result, setResult] = useState(null)
  const [best, setBest] = useState(0)
  const [round, setRound] = useState(0)
  const [returnTo, setReturnTo] = useState('home')
```

```

// Every screen starts at the top of the page.
useEffect(() => {
  window.scrollTo(0, 0)
}, [screen])

const goHome = useCallback(() => setScreen('home'), [])

const openLeaderboard = () => {
  setReturnTo(screen === 'report' ? 'report' : 'home')
  setScreen('leaderboard')
}

const pickCategory = (id) => {
  setCategoryId(id)
  setScreen('difficulty')
}

const navigate = (target) => {
  if (target === 'play') setScreen('category')
  else if (target === 'leaderboard') openLeaderboard()
  else setScreen(target)
}

const handleFinish = useCallback((res) => {
  const previousBest = getBestScore(res.categoryId, res.difficultyId)
  saveScore({
    name: getPlayerName() || 'Player',
    category: res.categoryId,
    difficulty: res.difficultyId,
    score: res.score,
    accuracy: res.accuracy,
    attempts: res.attempts,
    seconds: res.seconds,
    date: new Date().toISOString(),
  })
  setBest(previousBest)
  setResult(res)
  setScreen('report')
}, [])

const playAgain = () => {
  setRound((r) => r + 1)
  setScreen('game')
}

if (screen === 'splash') return <Splash onDone={goHome} />

const renderScreen = () => {
  switch (screen) {
    case 'home':
      return <Home onPlay={() => setScreen('category')} onPickCategory={pickCategory} />
    case 'howto':
      return <HowToPlay onBack={goHome} onPlay={() => setScreen('category')} />
    case 'category':
      return <CategorySelect onBack={goHome} onSelect={pickCategory} />
    case 'difficulty':
      return (
        <DifficultySelect
          categoryId={categoryId}
          onBack={() => setScreen('category')}
          onSelect={(id) => {
            setDifficultyId(id)
            setRound((r) => r + 1)
            setScreen('game')
          }}
        />
      )
    case 'game':
      return (

```

```

        <Game
          key={`${categoryId}-${difficultyId}-${round}`}
          categoryId={categoryId}
          difficultyId={difficultyId}
          onFinish={handleFinish}
          onQuit={() => setScreen('difficulty')}
        />
      )
    case 'report':
      return (
        <Report
          result={result}
          best={best}
          onPlayAgain={playAgain}
          onChangeMode={() => setScreen('category')}
          onHome={goHome}
          onLeaderboard={openLeaderboard}
        />
      )
    case 'leaderboard':
      return <Leaderboard onBack={() => setScreen(returnTo)} />
    default:
      return null
  }
}

return (
  <Shell active={NAV_SECTION[screen]} onNavigate={navigate}>
    {renderScreen()}
  </Shell>
)
}

```

File: src/data/science.js (the list of elements is shortened here)

```

// Science content used to build the card decks.
// Each category has 15 elements so the hardest grid (15 pairs) is always filled.
// Every element has a Lucide icon (shown on the card), a name, and a short fact
// shown in the popup once the pair is matched.

import {
  Dna, Brain, HeartPulse, Bone, Microscope, Bug, Eye, Leaf, Sprout, Egg, Fish, Feather,
  Stethoscope, Syringe, TreeDeciduous,
  Atom, Magnet, Zap, Lightbulb, Rainbow, Battery, Thermometer, Cog, Timer, Volume2, Scale,
  RadioTower, Plug, Apple, Waves,
  TestTube, FlaskConical, Droplet, Flame, Gem, Snowflake, Citrus, Radiation, Beaker,
  Pipette, Fuel, Coins, Hexagon, Cylinder, Droplets,
  Orbit, Rocket, Satellite, SatelliteDish, Telescope, Moon, Sun, Earth, Star, Sparkles,
  Aperture, Navigation, Cloud, Disc, MoonStar,
} from 'lucide-react'

// Each field has one colour, given as a ramp: strong, deep (text on tints), soft and tint
// (surfaces), line (borders).
export const CATEGORIES = [
  { id: 'biology', code: 'BIO', name: 'Biology', tagline: 'Cells, organs & living things',
    icon: Dna, color: '#059669', colorDeep: '#047857', colorSoft: '#ecfdf5', colorTint:
    '#d1fae5', colorLine: '#a7f3d0' },
  { id: 'physics', code: 'PHY', name: 'Physics', tagline: 'Forces, energy & matter', icon:
    Atom, color: '#2563eb', colorDeep: '#1d4ed8', colorSoft: '#eff6ff', colorTint: '#dbeafe',
    colorLine: '#bfdbfe' },
  { id: 'chemistry', code: 'CHM', name: 'Chemistry', tagline: 'Elements, reactions &
    compounds', icon: FlaskConical, color: '#d97706', colorDeep: '#b45309', colorSoft:
    '#fffbeb', colorTint: '#fef3c7', colorLine: '#fde68a' },
  { id: 'space', code: 'SPC', name: 'Space', tagline: 'Planets, stars & the cosmos', icon:
    Orbit, color: '#e11d48', colorDeep: '#be123c', colorSoft: '#fff1f2', colorTint: '#ffe4e6',
    colorLine: '#fecdd3' },
]

// Exposes a category's colour ramp to CSS. Spread into the `style` of any themed element.

```

```

export const themeStyle = (category) => ({
  '--accent': category.color,
  '--accent-deep': category.colorDeep,
  '--accent-soft': category.colorSoft,
  '--accent-tint': category.colorTint,
  '--accent-line': category.colorLine,
})

export const ELEMENTS = {
  biology: [
    { id: 'dna', name: 'DNA', icon: Dna, fact: 'DNA stores the genetic instructions for every living thing. Stretched out, the DNA in one human body would reach the Sun and back many times.' },
    { id: 'brain', name: 'Brain', icon: Brain, fact: 'The human brain has about 86 billion neurons and uses roughly 20% of the body\'s energy, even though it is only 2% of its weight.' },
    { id: 'heart', name: 'Heart', icon: HeartPulse, fact: 'The heart beats about 100,000 times a day, pumping roughly 7,500 litres of blood through the body.' },
    // ... 12 more elements in the same format
  ],
  physics: [
    { id: 'atom', name: 'Atom', icon: Atom, fact: 'An atom is the smallest unit of an element. It has a tiny dense nucleus of protons and neutrons, surrounded by electrons.' },
    { id: 'magnet', name: 'Magnet', icon: Magnet, fact: 'Every magnet has a north and a south pole. Cut one in half and you get two smaller magnets, each with both poles.' },
    { id: 'lightning', name: 'Lightning', icon: Zap, fact: 'A lightning bolt can heat the air around it to about 30,000 °C, five times hotter than the surface of the Sun.' },
    // ... 12 more elements in the same format
  ],
  chemistry: [
    { id: 'testtube', name: 'Test Tube', icon: TestTube, fact: 'Test tubes are made of borosilicate glass, which can be heated and cooled quickly without cracking.' },
    { id: 'flask', name: 'Flask', icon: FlaskConical, fact: 'The conical flask was designed by Emil Erlenmeyer in 1860. Its sloped sides let you swirl liquids without spilling.' },
    { id: 'water', name: 'Water', icon: Droplet, fact: 'Water is H2O: two hydrogen atoms bonded to one oxygen atom. It is the only common substance found naturally as solid, liquid and gas.' },
    // ... 12 more elements in the same format
  ],
  space: [
    { id: 'saturn', name: 'Saturn', icon: Orbit, fact: 'Saturn\'s rings are made mostly of ice and rock. Saturn is so light it would float in a bathtub big enough to hold it.' },
    { id: 'rocket', name: 'Rocket', icon: Rocket, fact: 'To escape Earth\'s gravity a rocket must reach about 11.2 km/s, which is known as escape velocity.' },
    { id: 'satellite', name: 'Satellite', icon: Satellite, fact: 'Sputnik 1, launched in 1957, was the first artificial satellite. Today thousands of satellites orbit Earth.' },
    // ... 12 more elements in the same format
  ],
}

// `columns` is the board width on phones, `columnsWide` on desktop layouts.
// `tone` is the signal colour used for the level badge.
export const DIFFICULTIES = [
  { id: 'easy', tone: 'emerald', name: 'Easy', pairs: 6, columns: 4, columnsWide: 4, peekMs: 3000, parSeconds: 45, multiplier: 1 },
  { id: 'medium', tone: 'amber', name: 'Medium', pairs: 8, columns: 4, columnsWide: 4, peekMs: 2500, parSeconds: 75, multiplier: 1.5 },
  { id: 'hard', tone: 'rose', name: 'Hard', pairs: 15, columns: 5, columnsWide: 6, peekMs: 2000, parSeconds: 150, multiplier: 2 },
]

export const getCategory = (id) => CATEGORIES.find((c) => c.id === id)
export const getDifficulty = (id) => DIFFICULTIES.find((d) => d.id === id)

```

File: src/utils/game.js

```
import { ELEMENTS } from '../data/science.js'
```

```

// Fisher-Yates shuffle, returns a new array.
export function shuffle(list) {
  const arr = [...list]
  for (let i = arr.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1))
    ;[arr[i], arr[j]] = [arr[j], arr[i]]
  }
  return arr
}

// Builds a shuffled deck of card objects for the given category and pair count.
export function buildDeck(categoryId, pairs) {
  const pool = shuffle(ELEMENTS[categoryId]).slice(0, pairs)
  const cards = pool.flatMap((element) => [
    { uid: `${element.id}-a`, element },
    { uid: `${element.id}-b`, element },
  ])
  return shuffle(cards)
}

// Accuracy is the share of attempts that produced a match.
export function calcAccuracy(matches, attempts) {
  if (attempts === 0) return 0
  return Math.round((matches / attempts) * 100)
}

// Score rewards matches, penalises wrong attempts and adds a bonus for beating par time.
export function calcScore({ pairs, attempts, seconds, difficulty }) {
  const mistakes = Math.max(0, attempts - pairs)
  const base = pairs * 100
  const penalty = mistakes * 15
  const timeBonus = Math.max(0, difficulty.parSeconds - seconds) * 3
  const raw = Math.max(0, base - penalty + timeBonus)
  return Math.round(raw * difficulty.multiplier)
}

// A friendly label describing how the player's memory performed, on a 1-5 scale.
export function memoryRating(accuracy) {
  if (accuracy >= 90) return { level: 5, label: 'Exceptional', note: 'Near-perfect recall. Your memory is razor sharp.' }
  if (accuracy >= 75) return { level: 4, label: 'Sharp', note: 'Very few slips. You are tracking positions really well.' }
  if (accuracy >= 60) return { level: 3, label: 'Good', note: 'Solid memory with a few guesses along the way.' }
  if (accuracy >= 45) return { level: 2, label: 'Developing', note: 'You are getting there. Try to focus on card positions.' }
  return { level: 1, label: 'Warming up', note: 'Keep practising. Memory improves fast with repetition.' }
}

export function formatTime(totalSeconds) {
  const m = Math.floor(totalSeconds / 60)
  const s = totalSeconds % 60
  return `${String(m).padStart(2, '0')}:${String(s).padStart(2, '0')}`
}

export function formatNumber(value) {
  return value.toLocaleString('en-US')
}

export function formatDate(iso) {
  const date = new Date(iso)
  if (Number.isNaN(date.getTime())) return '-'
  return date.toLocaleDateString('en-GB', { day: '2-digit', month: 'short' })
}

```

File: src/utils/storage.js

```
const SCORES_KEY = 'mindlab.scores'
```

```

const NAME_KEY = 'mindlab.playerName'
const MAX_ENTRIES = 50

function safeRead(key, fallback) {
  try {
    const raw = localStorage.getItem(key)
    return raw ? JSON.parse(raw) : fallback
  } catch {
    return fallback
  }
}

function safeWrite(key, value) {
  try {
    localStorage.setItem(key, JSON.stringify(value))
  } catch {
    // Storage may be unavailable in private mode; ignore.
  }
}

export function getPlayerName() {
  return safeRead(NAME_KEY, '') || ''
}

export function setPlayerName(name) {
  safeWrite(NAME_KEY, name.trim())
}

export function getScores() {
  const scores = safeRead(SCORES_KEY, [])
  return Array.isArray(scores) ? scores : []
}

export function saveScore(entry) {
  const scores = getScores()
  const record = { id: `${Date.now()}-${Math.random().toString(36).slice(2, 7)}`, ...entry }
  const next = [record, ...scores]
    .sort((a, b) => b.score - a.score || a.seconds - b.seconds)
    .slice(0, MAX_ENTRIES)
  safeWrite(SCORES_KEY, next)
  return record
}

export function clearScores() {
  safeWrite(SCORES_KEY, [])
}

// Best score for a category/difficulty combination, used on the report screen.
export function getBestScore(categoryId, difficultyId) {
  return getScores()
    .filter((s) => s.category === categoryId && s.difficulty === difficultyId)
    .reduce((best, s) => (s.score > best ? s.score : best), 0)
}

// Highest score per category, keyed by category id.
export function getBestByCategory() {
  const best = {}
  for (const s of getScores()) best[s.category] = Math.max(best[s.category] || 0, s.score)
  return best
}

```

File: src/hooks/useTimer.js

```

import { useEffect, useRef, useState } from 'react'

// Simple second-resolution timer. Starts when `running` becomes true.
export function useTimer(running) {
  const [seconds, setSeconds] = useState(0)

```

```

const startRef = useRef(null)

useEffect(() => {
  if (!running) return undefined
  if (startRef.current === null) startRef.current = Date.now()
  const tick = () => setSeconds(Math.floor((Date.now() - startRef.current) / 1000))
  tick()
  const id = setInterval(tick, 250)
  return () => clearInterval(id)
}, [running])

const reset = () => {
  startRef.current = null
  setSeconds(0)
}

return { seconds, reset }
}

```

File: src/components/Card.jsx

```

import { Check, Sparkle } from 'lucide-react'

export default function Card({ card, flipped, matched, disabled, onClick }) {
  const Icon = card.element.icon
  const classes = ['card', flipped && 'is-flipped', matched && 'is-matched'].filter(Boolean).join(' ')
  return (
    <button
      type="button"
      className={classes}
      onClick={onClick}
      disabled={disabled || matched || flipped}
      aria-label={flipped || matched ? card.element.name : 'Hidden card'}
    >
      <span className="card-inner">
        <span className="card-face card-back" aria-hidden="true">
          <span className="card-back-mark">
            <Sparkle size={11} strokeWidth={2} />
          </span>
        </span>
        <span className="card-face card-front">
          <span className="card-check" aria-hidden="true">
            <Check size={9} strokeWidth={3.5} />
          </span>
          <span className="card-icon">
            <Icon strokeWidth={1.5} />
          </span>
          <span className="card-name">{card.element.name}</span>
        </span>
      </span>
    </button>
  )
}

```

File: src/components/FactPopup.jsx

```

import { useEffect } from 'react'
import { createPortal } from 'react-dom'
import { ArrowRight, Check } from 'lucide-react'
import { ELEMENTS, themeStyle } from '../data/science.js'

// Fact card shown after a successful match, laid out like a small catalogue entry.
export default function FactPopup({ element, category, found, total, onClose }) {
  useEffect(() => {
    if (!element) return undefined
    const onKey = (e) => e.key === 'Escape' && onClose()
    window.addEventListener('keydown', onKey)
  })
}

```

```

    return () => window.removeEventListener('keydown', onKey)
  }, [element, onClose])

  if (!element) return null

  const Icon = element.icon
  const number = ELEMENTS[category.id].findIndex((e) => e.id === element.id) + 1
  const catalogRef = `${category.code}.${String(number).padStart(2, '0')}`

  return createPortal(
    <div className="popup-backdrop" onClick={onClose} role="presentation">
      <div
        className="popup"
        role="dialog"
        aria-modal="true"
        aria-labelledby="fact-title"
        onClick={(e) => e.stopPropagation()}
        style={themeStyle(category)}
      >
        <div className="popup-head">
          <span className="eyebrow">{catalogRef} / {category.name}</span>
          <span className="badge badge-emerald">
            <Check size={10} strokeWidth={3} /> Match
          </span>
        </div>

        <div className="popup-body">
          <span className="icon-tile icon-tile-lg icon-tile-solid">
            <Icon size={28} strokeWidth={1.6} />
          </span>
          <h2 id="fact-title" className="popup-title">{element.name}</h2>
          <p className="eyebrow">Did you know</p>
          <p className="popup-fact">{element.fact}</p>
        </div>

        <div className="popup-foot">
          <span className="eyebrow">Pair {found} of {total}</span>
          <button className="key key-solid" onClick={onClose} autoFocus>
            Continue <ArrowRight size={15} />
          </button>
        </div>
      </div>,
    document.body,
  )
}

```

File: src/components/StatsBar.jsx

```

import { formatTime } from '../utils/game.js'

export default function StatsBar({ seconds, attempts, matches, pairs, accuracy }) {
  const items = [
    { label: 'Time', value: formatTime(seconds) },
    { label: 'Pairs', value: `${matches}/${pairs}` },
    { label: 'Attempts', value: attempts },
    { label: 'Accuracy', value: `${accuracy}%` },
  ]

  return (
    <div className="panel stats-panel">
      <div className="stats">
        {items.map(({ label, value }) => (
          <div className="stat" key={label}>
            <span className="eyebrow stat-label">{label}</span>
            <span className="stat-value">{value}</span>
          </div>
        ))}
      </div>
    </div>
  )
}

```

```

        className="progress"
        role="progressbar"
        aria-label="Pairs found"
        aria-valuemin={0}
        aria-valuemax={pairs}
        aria-valuenow={matches}
      >
        <span style={{ width: `${(matches / pairs) * 100}%` }} />
      </div>
    </div>
  )
}

```

File: src/screens/Game.jsx

```

import { useCallback, useEffect, useMemo, useRef, useState } from 'react'
import { Eye, Layers, Pointer, RotateCcw } from 'lucide-react'
import TopBar from '../components/TopBar.jsx'
import Card from '../components/Card.jsx'
import FactPopup from '../components/FactPopup.jsx'
import StatsBar from '../components/StatsBar.jsx'
import { getCategory, getDifficulty, themeStyle } from '../data/science.js'
import { buildDeck, calcAccuracy, calcScore } from '../utils/game.js'
import { useTimer } from '../hooks/useTimer.js'

const MISMATCH_DELAY = 900

export default function Game({ categoryId, difficultyId, onFinish, onQuit }) {
  const category = getCategory(categoryId)
  const difficulty = getDifficulty(difficultyId)

  const [deck, setDeck] = useState(() => buildDeck(categoryId, difficulty.pairs))
  const [run, setRun] = useState(0) // bumps on restart so the peek bar
  replays
  const [flipped, setFlipped] = useState([]) // uids of currently face-up,
  unmatched cards
  const [matched, setMatched] = useState(() => new Set())
  const [found, setFound] = useState([]) // matched elements, in the order
  they were found
  const [attempts, setAttempts] = useState(0)
  const [peeking, setPeeking] = useState(true)
  const [locked, setLocked] = useState(false)
  const [fact, setFact] = useState(null)
  const [started, setStarted] = useState(false)
  const [finished, setFinished] = useState(false)

  const { seconds, reset: resetTimer } = useTimer(started && !finished)
  const finishedRef = useRef(false)

  const matches = matched.size / 2
  const remaining = difficulty.pairs - matches
  const accuracy = calcAccuracy(matches, attempts)

  // Show every card for a short peek at the start of the round.
  useEffect(() => {
    const id = setTimeout(() => setPeeking(false), difficulty.peekMs)
    return () => clearTimeout(id)
  }, [deck, difficulty.peekMs])

  const restart = useCallback(() => {
    finishedRef.current = false
    setDeck(buildDeck(categoryId, difficulty.pairs))
    setRun((r) => r + 1)
    setFlipped([])
    setMatched(new Set())
    setFound([])
    setAttempts(0)
    setPeeking(true)
    setLocked(false)
  }, [categoryId, difficulty.pairs])

```

```

    setFact(null)
    setStarted(false)
    setFinished(false)
    resetTimer()
  }, [categoryId, difficulty.pairs, resetTimer])

const handleFlip = (card) => {
  if (peeking || locked || finished) return
  if (flipped.includes(card.uid) || matched.has(card.uid)) return
  if (!started) setStarted(true)

  const next = [...flipped, card.uid]
  setFlipped(next)
  if (next.length < 2) return

  // Second card flipped: evaluate the attempt.
  setAttempts((a) => a + 1)
  setLocked(true)
  const [first, second] = next.map((uid) => deck.find((c) => c.uid === uid))
  const isMatch = first.element.id === second.element.id

  if (isMatch) {
    setMatched((prev) => new Set([...prev, first.uid, second.uid]))
    setFound((prev) => [...prev, first.element])
    setFlipped([])
    setFact(first.element)
  } else {
    setTimeout(() => {
      setFlipped([])
      setLocked(false)
    }, MISMATCH_DELAY)
  }
}

const closeFact = useCallback(() => {
  setFact(null)
  setLocked(false)
}, [])

// All pairs found: stop the clock and hand over the result.
useEffect(() => {
  if (finishedRef.current || matches !== difficulty.pairs || fact) return
  finishedRef.current = true
  setFinished(true)
  const score = calcScore({ pairs: difficulty.pairs, attempts, seconds, difficulty })
  onFinish({ categoryId, difficultyId, score, accuracy, attempts, seconds, pairs:
difficulty.pairs, found })
}, [matches, fact, difficulty, attempts, seconds, accuracy, categoryId, difficultyId,
found, onFinish])

// Board shape for phones and for wide layouts; CSS picks the right pair.
const boardStyle = useMemo(() => {
  const cards = difficulty.pairs * 2
  return {
    '--cols': difficulty.columns,
    '--rows': cards / difficulty.columns,
    '--cols-wide': difficulty.columnsWide,
    '--rows-wide': cards / difficulty.columnsWide,
  }
}, [difficulty])

return (
  <div className="view game themed" style={themeStyle(category)}>
    <div className="game-toolbar">
      <TopBar
        crumbs={[category.name, difficulty.name]}
        onBack={onQuit}
        right={

```

```

        <button className="key key-soft key-restart" onClick={restart} aria-
label="Restart round">
        <RotateCcw size={15} /> <span>Restart</span>
        </button>
    }
    />
</div>

    <StatsBar seconds={seconds} attempts={attempts} matches={matches}
pairs={difficulty.pairs} accuracy={accuracy} />

    <Status peeking={peeking} started={started} remaining={remaining}
peekMs={difficulty.peekMs} run={run} />

    <div className="stage dotgrid dotgrid-accent">
        <div className={`grid${difficulty.columns >= 5 ? ' is-dense' : ''}`}
style={boardStyle}>
            {deck.map((card) => (
                <Card
                    key={card.uid}
                    card={card}
                    flipped={peeking || flipped.includes(card.uid)}
                    matched={matched.has(card.uid)}
                    disabled={peeking || locked}
                    onClick={() => handleFlip(card)}
                />
            ))}
        </div>
    </div>

    <Discoveries found={found} total={difficulty.pairs} />

    <FactPopup element={fact} category={category} found={matches}
total={difficulty.pairs} onClose={closeFact} />
</div>
)
}

// One-line hint for the round. Always rendered so the layout never jumps.
function Status({ peeking, started, remaining, peekMs, run }) {
    if (peeking) {
        return (
            <div className="status is-peek" style={{ '--peek': `${peekMs}ms` }}>
                <Eye size={14} /> Memorise the cards
                <span className="status-bar" key={run} />
            </div>
        )
    }
    if (!started) {
        return (
            <div className="status">
                <Pointer size={14} /> Pick any card to begin
            </div>
        )
    }
    return (
        <div className="status">
            <Layers size={14} />
            {remaining === 0 ? 'All pairs found' : `${remaining} ${remaining === 1 ? 'pair' :
'pairs'} to go`}
        </div>
    )
}

// Running list of the facts unlocked this round, newest first.
function Discoveries({ found, total }) {
    return (
        <section className="panel log" aria-label="Discoveries">
            <header className="log-head">

```

```

    <span className="eyebrow">Discoveries</span>
    <span className="eyebrow">{found.length} / {total}</span>
  </header>
  {found.length === 0 ? (
    <p className="log-empty">Facts you unlock are collected here.</p>
  ) : (
    <ul className="log-list">
      {[...found].reverse().map((element) => {
        const Icon = element.icon
        return (
          <li className="log-item" key={element.id}>
            <span className="icon-tile icon-tile-sm">
              <Icon size={16} strokeWidth={1.75} />
            </span>
            <div>
              <div className="log-name">{element.name}</div>
              <div className="log-fact">{element.fact}</div>
            </div>
          </li>
        )
      })}
    </ul>
  )}
</section>
)
}

```

File: src/screens/Report.jsx

```

import { Clock, Crosshair, Layers, LayoutGrid, RotateCcw, Trophy, X } from 'lucide-react'
import TopBar from '../components/TopBar.jsx'
import PageHeader from '../components/PageHeader.jsx'
import ProgressRing from '../components/ProgressRing.jsx'
import { getCategory, getDifficulty, themeStyle } from '../data/science.js'
import { formatNumber, formatTime, memoryRating } from '../utils/game.js'
import { useCountUp } from '../hooks/useCountUp.js'

const LEVELS = [1, 2, 3, 4, 5]

export default function Report({ result, best, onPlayAgain, onChangeMode, onHome,
onLeaderboard }) {
  const category = getCategory(result.categoryId)
  const difficulty = getDifficulty(result.difficultyId)
  const rating = memoryRating(result.accuracy)
  const isNewBest = result.score > best
  const score = useCountUp(result.score)
  const accuracy = useCountUp(result.accuracy)
  const found = result.found || []

  const details = [
    { label: 'Completion time', value: formatTime(result.seconds), icon: Clock, tone:
'blue' },
    { label: 'Attempts', value: result.attempts, icon: Crosshair, tone: 'amber' },
    { label: 'Pairs matched', value: result.pairs, icon: Layers, tone: 'emerald' },
    { label: 'Mistakes', value: result.attempts - result.pairs, icon: X, tone: 'rose' },
  ]

  return (
    <div className="view report themed" style={themeStyle(category)}>
      <div className="report-crumbs">
        <TopBar crumbs={['Report', category.name, difficulty.name]} onBack={onHome} />
      </div>

      <div className="report-title">
        <PageHeader
          title={<>Round <span className="wavy">complete</span></>}
          sub={` ${category.name} on ${difficulty.name}, finished in $
{formatTime(result.seconds)}. `}
        />

```

```

</div>

<div className="report-actions">
  <button className="key key-solid key-lg" onClick={onPlayAgain}>
    <RotateCcw size={16} /> Play again
  </button>
  <button className="key key-soft key-lg" onClick={onChangeMode}>
    <LayoutGrid size={15} /> Change mode
  </button>
  <button className="key key-soft key-lg" onClick={onLeaderboard}>
    <Trophy size={15} /> Leaderboard
  </button>
</div>

<div className="report-grid">
  <div className="panel result-card">
    <ProgressRing value={accuracy} color={category.color}>
      <span className="ring-value">{accuracy}%</span>
      <span className="eyebrow ring-label">Accuracy</span>
    </ProgressRing>
    <div className="result-score">
      <p className="eyebrow">Score</p>
      <p className="result-score-value">{formatNumber(score)}</p>
      <p className="result-score-label">
        points
        {isNewBest && <span className="badge badge-rose">New best</span>}
      </p>
    </div>
  </div>

  <div className="panel memory-card">
    <div className="memory-head">
      <div>
        <p className="eyebrow">Memory performance</p>
        <p className="memory-rating">{rating.label}</p>
      </div>
      <span className="eyebrow">Level {rating.level} / 5</span>
    </div>
    <div className="level" aria-hidden="true">
      {LEVELS.map((n) => (
        <span key={n} className={n <= rating.level ? 'is-on' : ''} style={{ '--i': n
}} />
      ))}
    </div>
    <p className="memory-note">{rating.note}</p>
  </div>

  <div className="panel details">
    {details.map(({ label, value, icon: Icon, tone }) => (
      <div className={`detail tone-${tone}`} key={label}>
        <span className="detail-label">
          <span className="tone-chip"><Icon size={12} strokeWidth={2.25} /></span>
          <span className="eyebrow">{label}</span>
        </span>
        <span className="detail-value">{value}</span>
      </div>
    ))}
  </div>

  </div>

  {found.length > 0 && (
    <section className="section report-recap">
      <div className="section-head">
        <h2 className="section-title">What you discovered</h2>
        <span className="eyebrow">{found.length} facts</span>
      </div>
      <ul className="fact-grid">
        {found.map((element) => {
          const Icon = element.icon

```

```

        return (
          <li className="panel fact-card" key={element.id}>
            <span className="icon-tile icon-tile-sm">
              <Icon size={16} strokeWidth={1.75} />
            </span>
            <div>
              <div className="log-name">{element.name}</div>
              <div className="log-fact">{element.fact}</div>
            </div>
          </li>
        )
      })
    </ul>
  </section>
)
</div>
}

```

File: src/screens/Leaderboard.jsx

```

import { useState } from 'react'
import { Trophy } from 'lucide-react'
import TopBar from '../components/TopBar.jsx'
import PageHeader from '../components/PageHeader.jsx'
import Segmented from '../components/Segmented.jsx'
import { CATEGORIES, DIFFICULTIES, getCategory, getDifficulty, themeStyle } from '../data/science.js'
import { getScores, clearScores } from '../utils/storage.js'
import { formatDate, formatNumber, formatTime } from '../utils/game.js'

const LEVEL_OPTIONS = [
  { value: 'all', label: 'All' },
  ...DIFFICULTIES.map((d) => ({ value: d.id, label: d.name })),
]

export default function Leaderboard({ onBack }) {
  const [scores, setScores] = useState(getScores)
  const [category, setCategory] = useState('all')
  const [difficulty, setDifficulty] = useState('all')

  const visible = scores
    .filter((s) => category === 'all' || s.category === category)
    .filter((s) => difficulty === 'all' || s.difficulty === difficulty)
    .slice(0, 10)

  const handleClear = () => {
    if (window.confirm('Clear all saved scores?')) {
      clearScores()
      setScores([])
    }
  }

  return (
    <div className="view">
      <TopBar
        crumbs={['Scores', 'This device']}
        onBack={onBack}
        right={scores.length > 0 && (
          <button className="text-btn" onClick={handleClear}>Clear scores</button>
        )}
      />
      <PageHeader
        title={<><span className="wavy">Leaderboard</span></>}
        sub="The ten best rounds saved on this device."
      />
      <div className="toolbar">

```

```

        <Segmented options={LEVEL_OPTIONS} value={difficulty} onChange={setDifficulty}
label="Difficulty" />
        <div className="chips" role="group" aria-label="Category">
          <button className={`chip${category === 'all' ? ' is-active' : ''}`} onClick={()
=> setCategory('all')}>
            All
          </button>
          {CATEGORIES.map(({ id, name }) => (
            <button
              key={id}
              className={`chip${category === id ? ' is-active' : ''}`}
              onClick={() => setCategory(id)}
            >
              {name}
            </button>
          ))}
        </div>
      </div>

      {visible.length === 0 ? (
        <div className="empty dotgrid">
          <span className="icon-tile">
            <Trophy size={20} strokeWidth={1.6} />
          </span>
          <p className="empty-title">No scores yet</p>
          <p className="empty-text">Play a round to get on the board.</p>
        </div>
      ) : (
        <div className="panel table-wrap">
          <table className="score-table">
            <thead>
              <tr>
                <th className="col-rank">#</th>
                <th>Player</th>
                <th className="col-wide">Field</th>
                <th className="col-wide">Level</th>
                <th className="col-wide num">Accuracy</th>
                <th className="col-wide num">Time</th>
                <th className="col-wide num">Date</th>
                <th className="num">Score</th>
              </tr>
            </thead>
            <tbody>
              {visible.map((s, i) => {
                const cat = getCategory(s.category)
                const level = getDifficulty(s.difficulty)
                const Icon = cat.icon
                return (
                  <tr key={s.id} style={themeStyle(cat)}>
                    <td className="col-rank">
                      <span className={`rank rank-${i + 1}`}>{i + 1}</span>
                    </td>
                    <td>
                      <div className="player">
                        <span className="icon-tile icon-tile-sm">
                          <Icon size={16} strokeWidth={1.75} />
                        </span>
                        <div className="player-text">
                          <div className="player-name">{s.name}</div>
                          <div className="player-meta">
                            {cat.name} · {level.name} · {s.accuracy}% ·
                            {formatTime(s.seconds)}
                          </div>
                        </div>
                      </div>
                    </td>
                    <td className="col-wide">
                      <span className="field-tag"><i />{cat.name}</span>
                    </td>

```

```

        <td className="col-wide">
            <span className={`pill tone-${level.tone}`}>{level.name}</span>
        </td>
        <td className="col-wide num">{s.accuracy}%</td>
        <td className="col-wide num">{formatTime(s.seconds)}</td>
        <td className="col-wide num">{formatDate(s.date)}</td>
        <td className="num score-value">{formatNumber(s.score)}</td>
    </tr>
    )
    }}}
</tbody>
</table>
</div>
    )
</div>
)
}

```

File: src/styles/index.css (design tokens only)

```

/* =====
MindLab design system
Neutral palette, editorial serif headings, mono eyebrow labels,
tactile keycap buttons and layered hairline shadows.

Layout is mobile-first and opens up at 620 / 720 / 900 / 1000px.
===== */

:root {
    /* Neutral scale */
    --n-0: #ffffff;
    --n-50: #fafafa;
    --n-100: #f5f5f5;
    --n-200: #e5e5e5;
    --n-300: #d4d4d4;
    --n-400: #a3a3a3;
    --n-500: #737373;
    --n-600: #525252;
    --n-700: #404040;
    --n-800: #262626;
    --n-900: #171717;
    --n-950: #0a0a0a;

    /* Brand palette: four flat colours, one per science field.
    The same four double as state colours (success, info, caution, alert).
    Each has a strong step, a deep step for text on tints, and two surface tints. */
    --emerald: #059669;
    --emerald-deep: #047857;
    --emerald-soft: #ecfdf5;
    --emerald-tint: #d1fae5;
    --blue: #2563eb;
    --blue-deep: #1d4ed8;
    --blue-soft: #eff6ff;
    --blue-tint: #dbeafe;
    --amber: #d97706;
    --amber-deep: #b45309;
    --amber-soft: #fffbeb;
    --amber-tint: #fef3c7;
    --rose: #e11d48;
    --rose-deep: #be123c;
    --rose-soft: #fff1f2;
    --rose-tint: #ffe4e6;

    /* Themed screens override these with a field's colour ramp (see themeStyle in
    science.js) */
    --accent: var(--n-900);
    --accent-deep: var(--n-950);
    --accent-soft: var(--n-100);
    --accent-tint: var(--n-200);

```

```

--accent-line: var(--n-200);

--font-sans: 'Geist', ui-sans-serif, system-ui, -apple-system, 'Segoe UI', sans-serif;
--font-serif: 'Instrument Serif', ui-serif, Georgia, 'Times New Roman', serif;
--font-mono: 'Geist Mono', ui-monospace, SFMono-Regular, Menlo, Consolas, monospace;

--ease: cubic-bezier(0.32, 0.72, 0, 1);
--ease-out: cubic-bezier(0.22, 1, 0.36, 1);

/* Page frame */
--nav-h: 56px;
--page-x: 16px;
--page-y: 20px;
--page-max: 1120px;

/* Keycap shadows: soft top light, lift below, recess at the bottom edge */
--key-dark: 0 1px 1px rgba(0, 0, 0, 0.35), 0 3px 6px rgba(0, 0, 0, 0.28), 0 8px 16px
rgba(0, 0, 0, 0.22),
    inset 0 1px 2px rgba(255, 255, 255, 0.14), inset 0 -3px 6px rgba(0, 0, 0, 0.55);
--key-dark-pressed: 0 1px 2px rgba(0, 0, 0, 0.25), inset 0 2px 6px rgba(0, 0, 0, 0.55),
    inset 0 -1px 1px rgba(255, 255, 255, 0.06);
--key-light: 0 1px 1px rgba(0, 0, 0, 0.08), 0 2px 4px rgba(0, 0, 0, 0.1), 0 6px 12px
rgba(0, 0, 0, 0.08),
    inset 0 1px 2px rgba(255, 255, 255, 0.35), inset 0 -2px 4px rgba(0, 0, 0, 0.08);
--key-light-pressed: 0 1px 1px rgba(0, 0, 0, 0.05), inset 0 1px 2px rgba(0, 0, 0, 0.08),
    inset 0 2px 4px rgba(0, 0, 0, 0.04), inset 0 -1px 2px rgba(0, 0, 0, 0.05);

/* Surface shadows: a hairline ring plus a soft float */
--panel: 0 0 0 1px rgba(0, 0, 0, 0.07), 0 1px 2px rgba(0, 0, 0, 0.04), 0 6px 16px -4px
rgba(0, 0, 0, 0.05);
--depth: 0 0 0 1px rgba(0, 0, 0, 0.06), 0 1px 1px rgba(0, 0, 0, 0.04), 0 4px 10px
rgba(0, 0, 0, 0.05),
    0 12px 28px rgba(0, 0, 0, 0.05), inset 0 1px 1px rgba(255, 255, 255, 0.75), inset 0
-1px 2px rgba(0, 0, 0, 0.04);
--depth-hover: 0 0 0 1px rgba(0, 0, 0, 0.08), 0 2px 4px rgba(0, 0, 0, 0.05), 0 8px 16px
rgba(0, 0, 0, 0.07),
    0 20px 40px rgba(0, 0, 0, 0.06), inset 0 1px 1px rgba(255, 255, 255, 0.8), inset 0
-1px 2px rgba(0, 0, 0, 0.05);
--depth-pressed: 0 0 0 1px rgba(0, 0, 0, 0.08), 0 1px 2px rgba(0, 0, 0, 0.04),
    inset 0 2px 4px rgba(0, 0, 0, 0.08), inset 0 1px 0 rgba(255, 255, 255, 0.6);
}

```

TESTING

12.1 Testing Strategy

Testing was carried out to confirm that the application works according to the requirements listed in Section 8.1 and to find errors before the project was completed. Testing was done at the levels shown in Table 12.1. Most of the test cases were automated with scripts, so that they could be repeated after every change. The scripts are kept in the `tests` folder of the project and are run with the commands `npm test` and `npm run test:system`.

Table 12.1: Levels of testing		
Level	What was tested	How
Unit testing	Individual functions: time format, accuracy, score, memory rating, deck building, shuffle and the storage functions.	The script <code>tests/unit.mjs</code> calls each function with known inputs and compares the result with the expected value.
Integration and system testing	Complete flows through the real user interface: preview, flipping, matching, popup, statistics, report, leaderboard and navigation.	The script <code>tests/system.mjs</code> controls the browser, clicks the interface like a player and checks what is displayed.
Responsive testing	Layout at desktop, laptop, tablet and phone sizes.	The system tests are repeated at five screen sizes.
Static analysis	The whole source code.	Oxlint is run on the <code>src</code> folder.
Build verification	The production build.	The command <code>npm run build</code> must finish without errors.

12.2 Test Environment

The tests were run in the environment described below.

Item	Details
Browser	Google Chrome 154
Operating system	macOS
Runtime for unit tests	Node.js 22.18.0
Screen sizes	1440 × 900, 1280 × 720, 820 × 1180, 390 × 844 and 375 × 667 pixels
Application build	Vite development server (version 8.3.3)

12.3 Unit Test Cases

Table 12.2 lists the unit test cases. Test cases U01 to U31 check the game functions and the science content, and S01 to S11 check the storage functions. 42 of 42 test cases passed.

Table 12.2: Unit test cases and results				
ID	Test case	Expected result	Actual result	Status
U01	formatTime(0)	00:00	00:00	Pass
U02	formatTime(65)	01:05	01:05	Pass
U03	formatTime(600)	10:00	10:00	Pass
U04	calcAccuracy(0 matches, 0 attempts)	0	0	Pass
U05	calcAccuracy(6, 6)	100	100	Pass
U06	calcAccuracy(6, 7)	86	86	Pass
U07	calcAccuracy(15, 18)	83	83	Pass
U08	calcScore Easy: 6 pairs, 7 attempts, 16 s	672	672	Pass
U09	calcScore Medium: 8 pairs, 10 attempts, 23 s	1389	1389	Pass
U10	calcScore Hard: 15 pairs, 18 attempts, 42 s	3558	3558	Pass
U11	calcScore with no time bonus (Easy, 6 attempts, 200 s)	600	600	Pass
U12	calcScore never goes below zero (Easy, 60 attempts, 100 s)	0	0	Pass
U13	memoryRating(90)	Exceptional (5)	Exceptional (5)	Pass
U14	memoryRating(89)	Sharp (4)	Sharp (4)	Pass
U15	memoryRating(75)	Sharp (4)	Sharp (4)	Pass
U16	memoryRating(74)	Good (3)	Good (3)	Pass
U17	memoryRating(60)	Good (3)	Good (3)	Pass
U18	memoryRating(59)	Developing (2)	Developing (2)	Pass
U19	memoryRating(45)	Developing (2)	Developing (2)	Pass
U20	memoryRating(44)	Warming up (1)	Warming up (1)	Pass
U21	buildDeck(physics, 6) card count	12	12	Pass
U22	buildDeck: every element appears exactly twice	True	True	Pass
U23	buildDeck: card ids are unique	12	12	Pass
U24	buildDeck: all cards belong to the chosen category	True	True	Pass
U25	buildDeck(space, 15) card count and distinct elements	30 cards, 15 elements	30 cards, 15 elements	Pass

Table 12.2: Unit test cases and results				
ID	Test case	Expected result	Actual result	Status
U26	shuffle keeps the same members and leaves the input unchanged	True	True	Pass
U27	buildDeck gives different orders across 40 runs	True	True	Pass
U28	formatNumber(3558)	3,558	3,558	Pass
U29	Content: 4 categories with 15 elements each	15,15,15,15	15,15,15,15	Pass
U30	Content: every element has a name, icon and fact	True	True	Pass
U31	Content: element ids are unique inside each category	True	True	Pass
S01	getScores() on empty storage	Empty list	Empty list	Pass
S02	getPlayerName() on empty storage	Empty text	Empty text	Pass
S03	setPlayerName trims spaces	Indu	Indu	Pass
S04	saveScore keeps scores sorted high to low, faster time first on a tie	900/20 900/40 700/25 500/30	900/20 900/40 700/25 500/30	Pass
S05	getBestScore(physics, easy)	900	900	Pass
S06	getBestScore for a mode never played	0	0	Pass
S07	getBestByCategory()	physics: 900, space: 700	physics: 900, space: 700	Pass
S08	storage keeps only the best 50 records	50	50	Pass
S09	lowest scores are the ones dropped	114	114	Pass
S10	clearScores() empties the list	0	0	Pass
S11	corrupted storage is read as an empty list	Empty list	Empty list	Pass

12.4 System Test Cases

Table 12.3 lists the system test cases, which were run on the real interface in the browser. 43 of 43 test cases passed.

Table 12.3: System test cases and results				
ID	Test case	Expected result	Actual result	Status
F01	Splash screen is shown when the app opens	True	True	Pass
F02	Home page opens after the splash	True	True	Pass
F03	Player name field accepts at most 18 characters	18	18	Pass
F04	Home lists all four science categories	Biology, Physics, Chemistry, Space	Biology, Physics, Chemistry, Space	Pass
F05	Easy board deals 12 cards	12	12	Pass

Table 12.3: System test cases and results

ID	Test case	Expected result	Actual result	Status
F06	All cards are face up during the peek	12	12	Pass
F07	Cards cannot be clicked during the peek	12	12	Pass
F08	All cards are hidden when the peek ends	0	0	Pass
F09	Timer does not run before the first flip	00:00	00:00	Pass
F10	First flip turns one card and is not counted as an attempt	1 flipped, 0 attempts	1 flipped, 0 attempts	Pass
F11	Two different cards count as one attempt with 0% accuracy	1 attempts, 0%	1 attempts, 0%	Pass
F12	A third card cannot be flipped while two are showing	2	2	Pass
F13	Unmatched cards flip back face down	0	0	Pass
F14	A matching pair opens the fact popup for that element	Tree	Tree	Pass
F15	The popup shows a science fact	True	True	Pass
F16	Matched pair is counted (pairs 1/6, 2 attempts, 50%)	1/6,2,50%	1/6,2,50%	Pass
F17	Board is locked while the fact popup is open	0	0	Pass
F18	Clicking a matched card does nothing	0 flipped, 2 attempts	0 flipped, 2 attempts	Pass
F19	The matched element is added to the discoveries list	Tree	Tree	Pass
F20	Timer runs after the first flip	True	True	Pass
F21	Restart resets time, pairs, attempts and accuracy	00:00,0/6,0,0%	00:00,0/6,0,0%	Pass
F22	Restart starts a new peek	True	True	Pass
F23	Report opens when the last pair is matched	True	True	Pass
F24	Score on the report equals the scoring formula	726	726	Pass
F25	Mistakes = attempts - pairs	0	0	Pass
F26	Accuracy on the report = pairs / attempts	100%	100%	Pass
F27	First result in a mode is marked as a new best	True	True	Pass
F28	Report lists one fact for every pair found	6	6	Pass
F29	A lower score in the same mode is not marked as a new best	Score is lower and no badge is shown	Score is lower and no badge is shown	Pass
F30	Leaderboard lists both rounds, highest score first	726,633	726,633	Pass
F31	A blank name is saved as "Player"	Player	Player	Pass
F32	Filtering by a category with no scores shows the empty state	No scores yet	No scores yet	Pass

Table 12.3: System test cases and results

ID	Test case	Expected result	Actual result	Status
F33	Filtering by Easy shows the two Easy rounds	2	2	Pass
F34	Filtering by Hard shows no rounds	True	True	Pass
F35	Scores are still there after the page is reloaded	2	2	Pass
F36	Home shows rounds played and best score after reload	2,726	2,726	Pass
F37	Clear scores asks for confirmation, then empties the board	1 dialog, 0 scores	1 dialog, 0 scores	Pass
F38	Hard board deals 30 cards in 6 columns on desktop	30 cards, 6 columns	30 cards, 6 columns	Pass
F39	Desktop game fits the window without page scrolling	True	True	Pass
F40	Back button steps Game > Difficulty > Category > Home	Difficulty, then Category, then Home	Difficulty, then Category, then Home	Pass
F41	How to play page shows six steps	6	6	Pass
F42	No sideways scrolling and the Hard board stays in view at five screen sizes	No sideways scrolling and board in view at 1440×900, 1280×720, 820×1180, 390×844 and 375×667	As expected at all five sizes (6, 6, 5, 5 and 5 columns)	Pass
F43	Phone navigation shows icons only	none	none	Pass

12.5 Validation of Scoring with Complete Rounds

As a final check, the scores of five complete rounds played in the application were compared with scores calculated by hand from the formula in Module 7. The results are given in Table 12.4. In every round, the two values are equal.

Table 12.4: Scores of complete rounds compared with the formula

Category	Level	Pairs	Attempts	Time (s)	Calculation by formula	Score shown
Chemistry	Easy	6	7	16	$(600 - 15 + 87) \times 1$	672
Space	Hard	15	18	42	$(1500 - 45 + 324) \times 2$	3,558
Biology	Medium	8	10	23	$(800 - 30 + 156) \times 1.5$	1,389
Physics	Medium	8	9	22	$(800 - 15 + 159) \times 1.5$	1,416
Biology	Easy	6	6	13	$(600 - 0 + 96) \times 1$	696

12.6 Test Summary

Table 12.5: Summary of test results			
Type of test	Test cases	Passed	Failed
Unit tests	42	42	0
System tests	43	43	0
Scoring validation rounds	5	5	0
Static analysis (Oxlint)	Whole source	0 errors, 0 warnings	–
Production build	1	1	0

All the test cases passed, and no error was open at the end of testing. It should be noted that the tests were run in Google Chrome only. Testing in other browsers is listed as a limitation in Chapter 13.

FUTURE SCOPE

13.1 Limitations of the Present System

- The leaderboard is stored on one device. Scores cannot be compared between players on different devices, and they are lost if the browser data is cleared.
- The science content is fixed at 60 elements in four categories and is available only in English.
- The cards show icons and names. Real photographs and diagrams are not used.
- The game has no sound.
- There are no user accounts, so two players who type the same name are treated as one player.
- The application was tested in Google Chrome. It should work in other modern browsers, but this was not tested during the project.

13.2 Future Enhancements

The project can be extended in the following directions.

1. **Online leaderboard.** A server and a database can be added so that scores are shared between all players, with user registration and login. The ER diagram in Section 8.6 can be used directly as the database design.
2. **More content.** New categories such as Mathematics, Geography and Computer Science, and more elements in each category, can be added in the data file.
3. **Syllabus-based levels.** The content can be arranged class-wise, so that a school can use the game for a particular chapter.
4. **Quiz mode.** After a round, a short quiz on the facts just shown would check whether the player remembers them.
5. **Pictures and sound.** Real images, sound effects and spoken facts would make the game richer and more accessible.

6. **Multiplayer mode.** Two players could take turns on the same board, or compete online in real time.
7. **Regional languages.** The facts can be translated into Hindi and other languages.
8. **Installable and offline application.** By converting it into a Progressive Web App, the game could be installed on a phone and played without internet.
9. **Teacher dashboard.** Teachers could see the progress of each student and the facts that students find difficult.
10. **Adaptive difficulty.** The game could select the level automatically from the past accuracy of the player.

13.3 Conclusion

The project MindLab – Science Memory Challenge achieved its objectives. A complete web application was developed in which a player can choose a science field and a level, play a memory round, learn a fact with every matched pair, view a detailed performance report and compare scores on a leaderboard. The application works on desktop, tablet and mobile screens and needs no installation or login.

The project gave practical experience of the full development cycle: analysing requirements, designing with data flow and ER diagrams, building a component-based user interface, writing game logic, storing data in the browser and testing the result in a systematic way. It shows that a simple game, when combined with meaningful content, can become a useful tool for learning.

REFERENCES

Books

1. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.
2. R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson Education.
3. D. E. Knuth, The Art of Computer Programming, Volume 2: Seminumerical Algorithms, Addison-Wesley. (Shuffling algorithm)

Websites

1. React documentation: <https://react.dev>
2. Vite documentation: <https://vite.dev>
3. MDN Web Docs (JavaScript, CSS and the Web Storage API): <https://developer.mozilla.org>
4. Lucide icon library: <https://lucide.dev>
5. Google Fonts (Geist, Geist Mono and Instrument Serif): <https://fonts.google.com>
6. Oxcint documentation: <https://oxc.rs>
7. Node.js documentation: <https://nodejs.org>
8. Opensource UI component library, used as a visual design reference: <https://opensourceui.in>
9. Source code of this project: <https://github.com/Indu0213/mind-lab>

All websites were accessed in October 2026.